

```
In [2]: #!/jupyter nbconvert --to html /content/Yulu.ipynb
```

Delhivery — Case Study

by Suhel Mandal

Concept Used:

- Feature Creation
- Relationship between Features
- Column Normalization /Column Standardization
- Handling categorical values
- Missing values - Outlier treatment / Types of outliers

Table of Contents:

1. Importing th Libraries
2. Loading the Data
3. Exploring the Data (EDA)
4. Handling Missing Values
5. Converting Date Columns
6. Creating New Features
7. Merging Rows — Segment Level
8. Merging Rows — Trip Level
9. Hypothesis Testing
10. Finding and Fixing Outliers
11. One-Hot Encoding
12. Scaling / Normalisation
13. Business Insights & Recommendations

1.Importing the libraries

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

```
from scipy.stats import ttest_1samp
from scipy.stats import ttest_ind
from scipy.stats import ttest_rel
from scipy.stats import chisquare
from scipy.stats import chi2
from scipy.stats import chi2_contingency

from statsmodels.stats.proportion import proportions_ztest
from statsmodels.graphics.gofplots import qqplot

from scipy.stats import f_oneway # Numeric Vs categorical for many
from scipy.stats import kruskal # Numeric Vs categorical for many c
from scipy.stats import shapiro
from scipy.stats import levene
from scipy.stats import kstest

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler

from IPython.display import HTML
import warnings
warnings.filterwarnings('ignore')

# style for all plots
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)
```

2.Loading the Data

```
In [4]: url = "https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/0
df = pd.read_csv(url)
```

```
In [5]: df.head()
```

Out[5]:

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	15374109

5 rows x 24 columns

```
In [6]: print('Dataset Loaded')
print(f'Number of Rows: {df.shape[0]}')
print(f'Number of Columns: {df.shape[1]}')
```

Dataset Loaded
Number of Rows: 144867
Number of Columns: 24



Observations:

- The dataset was successfully loaded into the dataframe.
- The dataset contains multiple rows and columns related to logistics operations.
- Each row represents a segment of a delivery trip.
- A single trip may contain multiple source-destination movements.
- The dataset includes:
 - Time-related features
 - Distance-related features
 - Route information
 - Segment-level delivery details
- This data can be used for:
 - Feature engineering
 - Route optimization
 - Delivery performance analysis
 - Forecasting models

3.Exploring the Data (EDA)

Before doing anything, lets explore what the data looks like

```
In [7]: # all columns
print(f'The Columns present in the dataset are: \n{df.columns.to_li
```

The Columns present in the dataset are:
['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type', 'trip_uuid', 'source_center', 'source_name', 'destination_center', 'destination_name', 'od_start_time', 'od_end_time', 'start_scan_to_end_scan', 'is_cutoff', 'cutoff_factor', 'cutoff_timestamp', 'actual_distance_to_destination', 'actual_time', 'osrm_time', 'osrm_distance', 'factor', 'segment_actual_time', 'segment_osrm_time', 'segment_osrm_distance', 'segment_factor']

```
In [8]: # data types
display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   data                                     144867 non-null  object
1   trip_creation_time                     144867 non-null  object
2   route_schedule_uuid                   144867 non-null  object
3   route_type                             144867 non-null  object
4   trip_uuid                             144867 non-null  object
5   source_center                         144867 non-null  object
6   source_name                           144574 non-null  object
7   destination_center                    144867 non-null  object
8   destination_name                      144606 non-null  object
9   od_start_time                         144867 non-null  object
10  od_end_time                           144867 non-null  object
11  start_scan_to_end_scan                 144867 non-null  float64
12  is_cutoff                             144867 non-null  bool
13  cutoff_factor                         144867 non-null  int64
14  cutoff_timestamp                      144867 non-null  object
15  actual_distance_to_destination          144867 non-null  float64
16  actual_time                           144867 non-null  float64
17  osrm_time                             144867 non-null  float64
18  osrm_distance                         144867 non-null  float64
19  factor                                144867 non-null  float64
20  segment_actual_time                   144867 non-null  float64
21  segment_osrm_time                     144867 non-null  float64
22  segment_osrm_distance                 144867 non-null  float64
23  segment_factor                        144867 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB
None
```

```
In [9]: # ststistical summary of number columns
```

```
display(df.describe(include='all').T)
```

	count	unique	top	
data	144867	2	training	104
trip_creation_time	144867	14817	2018-09-22 04:55:04.835022	
route_schedule_uuid	144867	1504	thanos::sroute:4029a8a2- 6c74-4b7e-a6d8- f9e069f...	
route_type	144867	2	FTL	99
trip_uuid	144867	14817	trip- 153759210483476123	
source_center	144867	1508	IND000000ACB	23
source_name	144574	1498	Gurgaon_Bilaspur_HB (Haryana)	23
destination_center	144867	1481	IND000000ACB	15
destination_name	144606	1468	Gurgaon_Bilaspur_HB (Haryana)	15
od_start_time	144867	26369	2018-09-21 18:37:09.322207	
od_end_time	144867	26369	2018-09-24 09:59:15.691618	
start_scan_to_end_scan	144867.0	NaN	NaN	
is_cutoff	144867	2	True	118
cutoff_factor	144867.0	NaN	NaN	
cutoff_timestamp	144867	93180	2018-09-24 05:19:20	
actual_distance_to_destination	144867.0	NaN	NaN	
actual_time	144867.0	NaN	NaN	
osrm_time	144867.0	NaN	NaN	
osrm_distance	144867.0	NaN	NaN	
factor	144867.0	NaN	NaN	
segment_actual_time	144867.0	NaN	NaN	
segment_osrm_time	144867.0	NaN	NaN	
segment_osrm_distance	144867.0	NaN	NaN	
segment_factor	144867.0	NaN	NaN	

```
In [10]: # finding duplicate rows
# Number of duplicate rows
duplicate_rows_count = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_rows_count}")

# Displaying the duplicate rows
if duplicate_rows_count > 0:
    print("\nDuplicate rows:")
    display(df[df.duplicated()])
else:
    print("No duplicate rows found.")
```

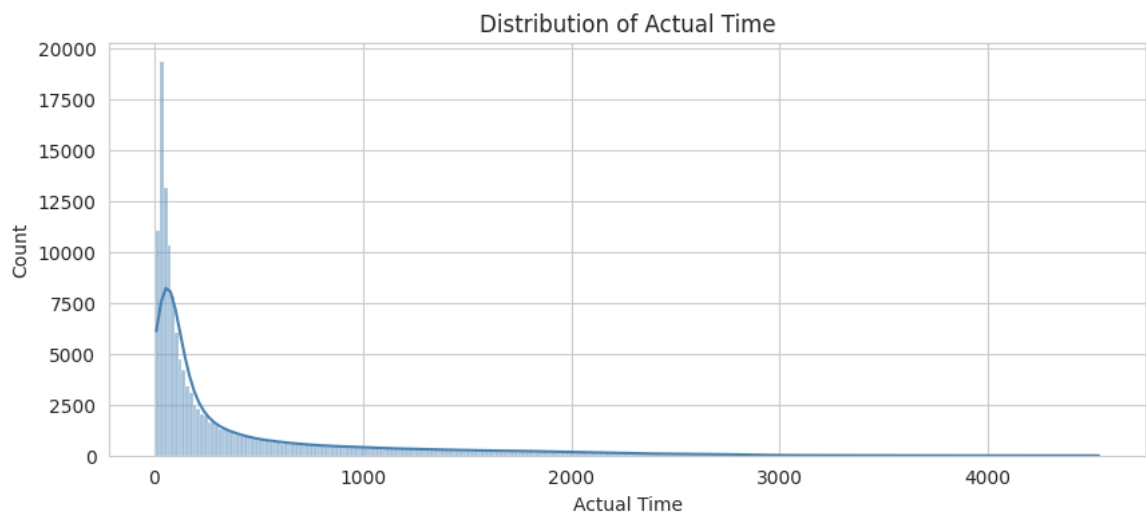
Number of duplicate rows: 0
No duplicate rows found.

Distribution Plots

```
In [11]: # These show us how the values are spread out.

# Histogram for actual time taken
plt.figure(figsize=(10, 4))
sns.histplot(df['actual_time'].dropna(), kde=True, color='steelblue')
plt.title('Distribution of Actual Time')
plt.xlabel('Actual Time')
plt.show()

# INSIGHT: Right-skewed – most deliveries finish quickly, but some
```



```
In [64]: print(df['actual_time'].describe())
print("Skewness:", round(df['actual_time'].skew(), 2))
```

```
count      144867.000000
mean       416.927527
std        598.103621
min         9.000000
25%        51.000000
50%       132.000000
75%       513.000000
max      4532.000000
Name: actual_time, dtype: float64
Skewness: 2.07
```

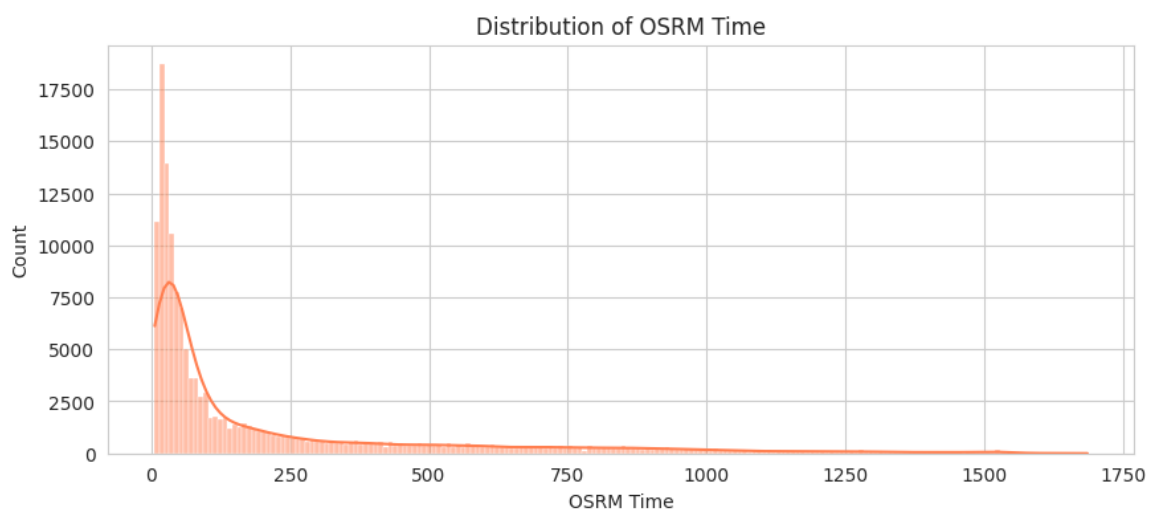


Observations:

- Business Meaning
 - The fact that most deliveries are quick is a good sign for Delhivery's operations.
 - However, the extreme values on the right are a concern — they represent poor customer experience and need to be investigated
 - A right-skewed distribution is very typical in logistics data worldwide
 - The actual_time distribution is right-skewed with a mean greater than the median, indicating the presence of high-value outliers that inflate the average. These will be treated using the IQR method.

```
In [12]: # Histogram for osrm time
```

```
plt.figure(figsize=(10,4))
sns.histplot(df['osrm_time'].dropna(), kde = True, color = 'coral')
plt.title('Distribution of OSRM Time')
plt.xlabel('OSRM Time')
plt.show()
```



```
In [65]: print(df['osrm_time'].describe())
print("Skewness:", round(df['osrm_time'].skew(), 2))
```

```
count      144867.000000
mean       213.868272
std        308.011085
min         6.000000
25%        27.000000
50%        64.000000
75%       257.000000
max       1686.000000
Name: osrm_time, dtype: float64
Skewness: 2.05
```

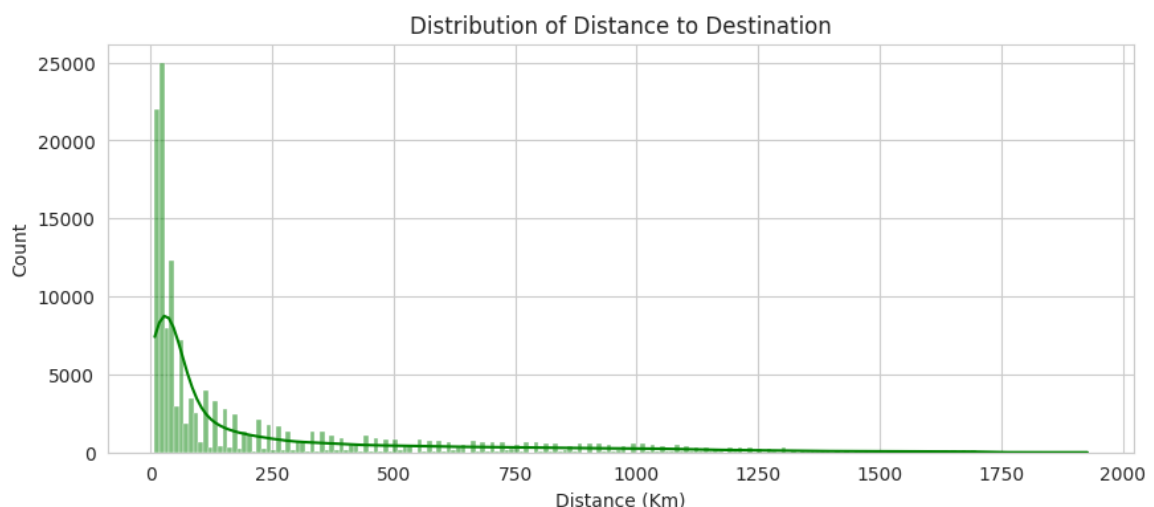


Observations:

- Business Meaning
 - OSRM is a useful baseline estimate but should never be shown directly to customers as the promised delivery time
 - Delivery should add a buffer on top of OSRM time when communicating ETAs
 - OSRM time follows a similar right-skewed distribution as actual time, but with lower values overall. The consistent gap between the two suggests that OSRM systematically underestimates delivery time, and a correction factor should be applied before using it for customer-facing ETA predictions.

```
In [13]: # Histogram for actual distance to destination

plt.figure(figsize=(10,4))
sns.histplot(df['actual_distance_to_destination'].dropna(), kde = T
plt.title('Distribution of Distance to Destination')
plt.xlabel('Distance (Km)')
plt.show()
```



```
In [66]: print(df['actual_distance_to_destination'].describe())
print("Skewness:", round(df['actual_distance_to_destination'].skew(
```



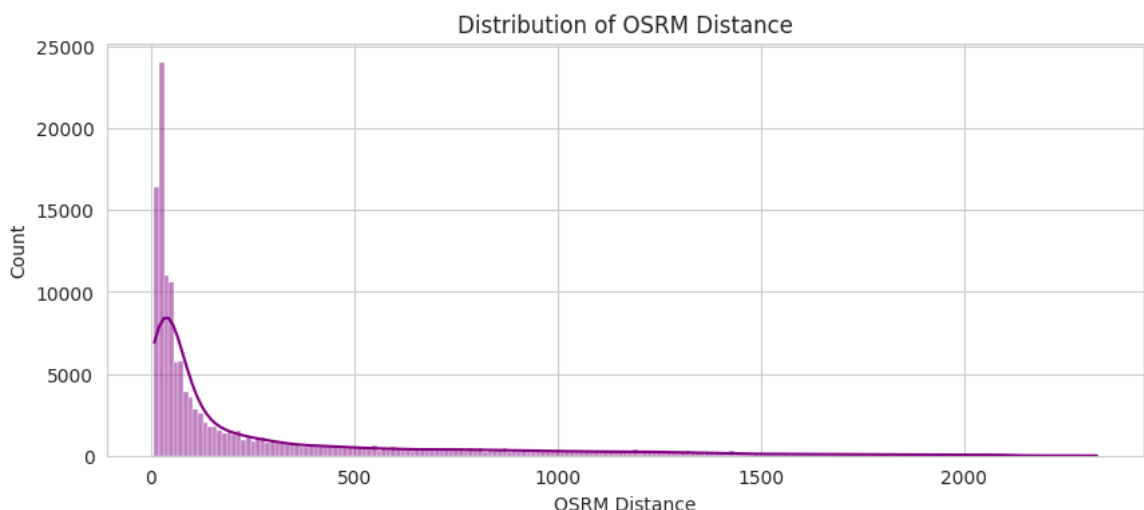
```
count      144867.000000
mean       234.073372
std        344.990009
min         9.000045
25%        23.355874
50%        66.126571
75%       286.708875
max       1927.447705
Name: actual_distance_to_destination, dtype: float64
Skewness: 1.99
```

Observations:

- Business Meaning
 - The concentration of short distances means Delhivery's core business is regional delivery, not just long haul
 - Long distance outliers cost more to operate and take longer — they need special planning and pricing
 - Understanding the distance distribution helps in fleet sizing — more small vehicles for short routes, fewer large trucks for long ones
 - The actual distance to destination is right-skewed, with most deliveries covering short to medium distances. The long right tail suggests a small proportion of long-haul deliveries which are likely FTL route type trips, as confirmed by the route type boxplot seen earlier.

```
In [14]: # Histogram for osrm distance
```

```
plt.figure(figsize=(10, 4))
sns.histplot(df['osrm_distance'].dropna(), kde=True, color='purple')
plt.title('Distribution of OSRM Distance')
plt.xlabel('OSRM Distance')
plt.show()
```



```
In [67]: print(df['osrm_distance'].describe())
print("Skewness:", round(df['osrm_distance'].skew(), 2))
```

```
count      144867.000000
mean        284.771297
std         421.119294
min          9.008200
25%         29.914700
50%         78.525800
75%        343.193250
max       2326.199100
Name: osrm_distance, dtype: float64
Skewness: 2.05
```

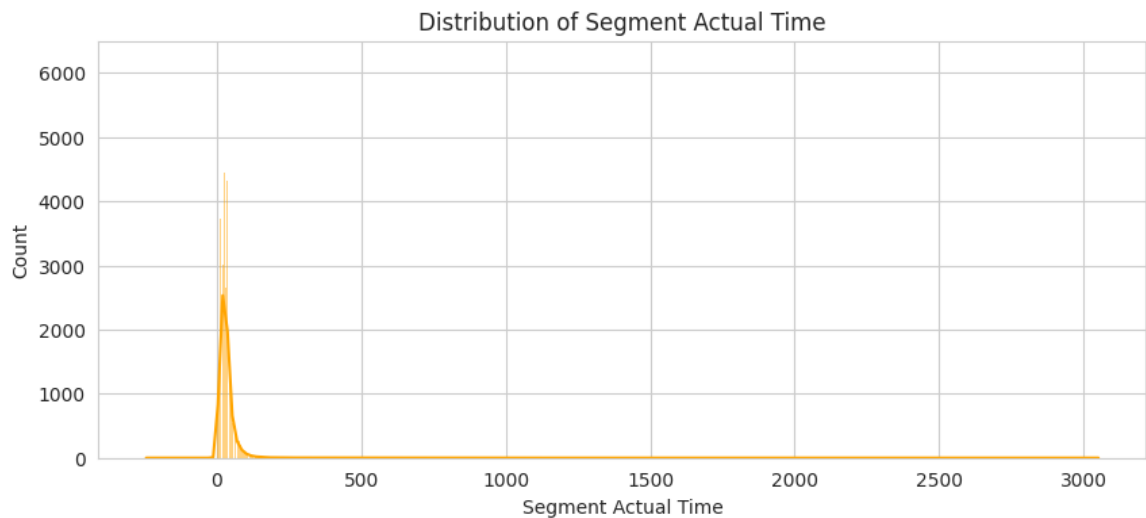


Observations:

- Business Meaning
 - If actual distance is consistently higher than OSRM distance across most trips, it means drivers are not taking optimal routes
 - This directly increases fuel costs and delivery time
 - Delhivery could use this gap to identify routes where drivers need better navigation guidance
 - The OSRM distance distribution closely mirrors the actual distance distribution in shape, both being right-skewed. However actual distance values tend to be slightly higher than OSRM distance, suggesting that real-world routes are longer than the theoretically optimal path — pointing to inefficiencies in route adherence that directly impact fuel costs and delivery time.

```
In [15]: # Histogram for segment actual time

plt.figure(figsize=(10, 4))
sns.histplot(df['segment_actual_time'].dropna(), kde=True, color='o')
plt.title('Distribution of Segment Actual Time')
plt.xlabel('Segment Actual Time')
plt.show()
```



```
In [16]: print(df['segment_actual_time'].describe())
print("Skewness:", df['segment_actual_time'].skew())
```

```
count      144867.000000
mean         36.196111
std         53.571158
min        -244.000000
25%         20.000000
50%         29.000000
75%         40.000000
max         3051.000000
Name: segment_actual_time, dtype: float64
Skewness: 16.827413001372534
```



Observations — Segment Actual Time

1. Range & Central Tendency

The minimum value is -244.000000 and maximum is 3051. The mean is around 36.196111 and median is 29. Since mean > median, the distribution is right-skewed (most common pattern in logistics data).

2. Shape of Distribution

The distribution is right-skewed — most segments are completed in a short time, but a small number of segments take much longer.

3. Outliers

The long right tail suggests there are outliers — segments that took unusually long, possibly due to traffic, loading delays, or bad routes.

4. Business Meaning

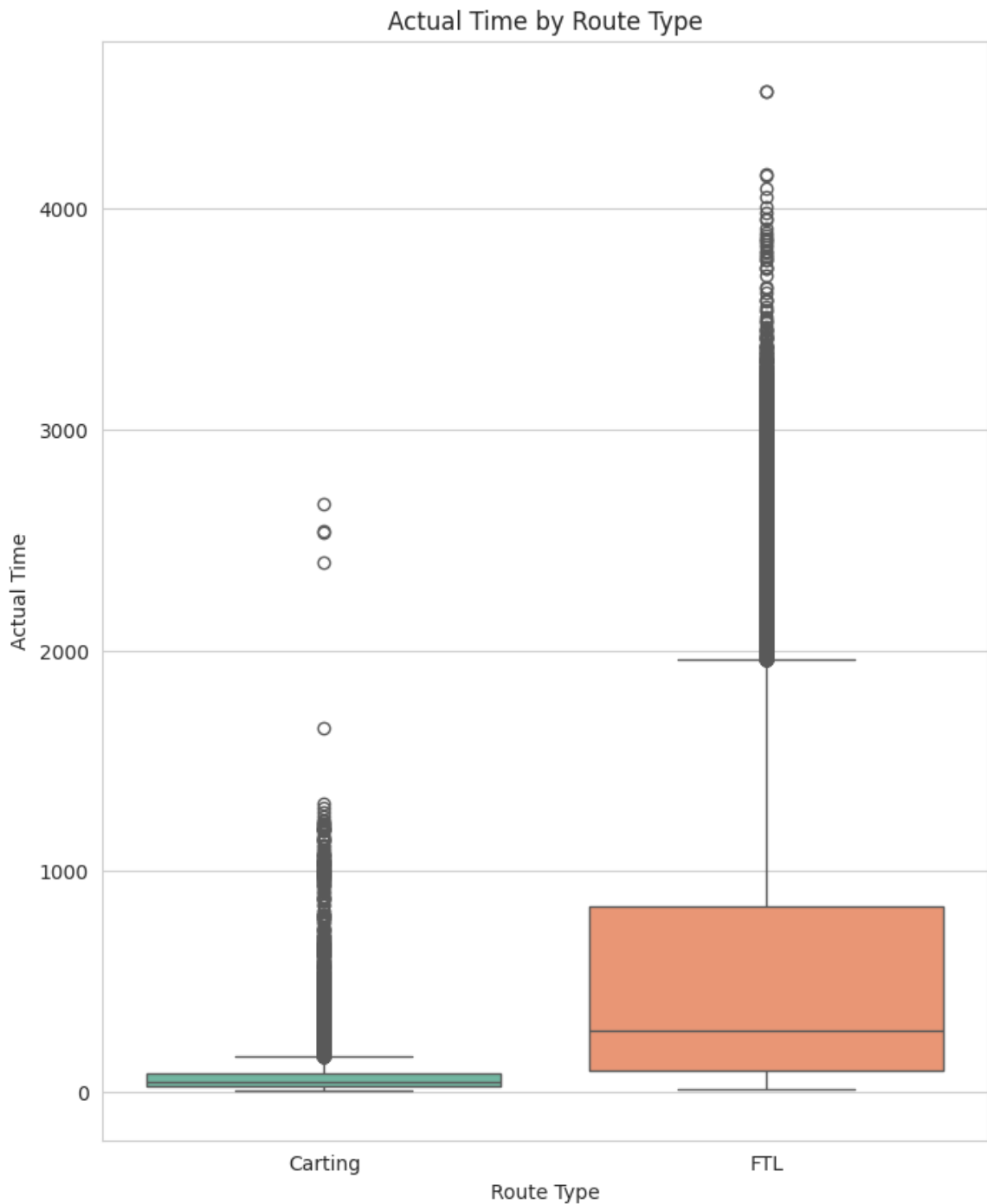
segment_actual_time measures one leg of a journey (e.g. Delhi → Jaipur), not the full trip. Short segments dominate, which makes sense — most hub-to-hub transfers are nearby. The few very long segments could

indicate inefficient routes or problem deliveries worth investigating

```
In [17]: # Boxplot: Comparing actual delivery time by route type

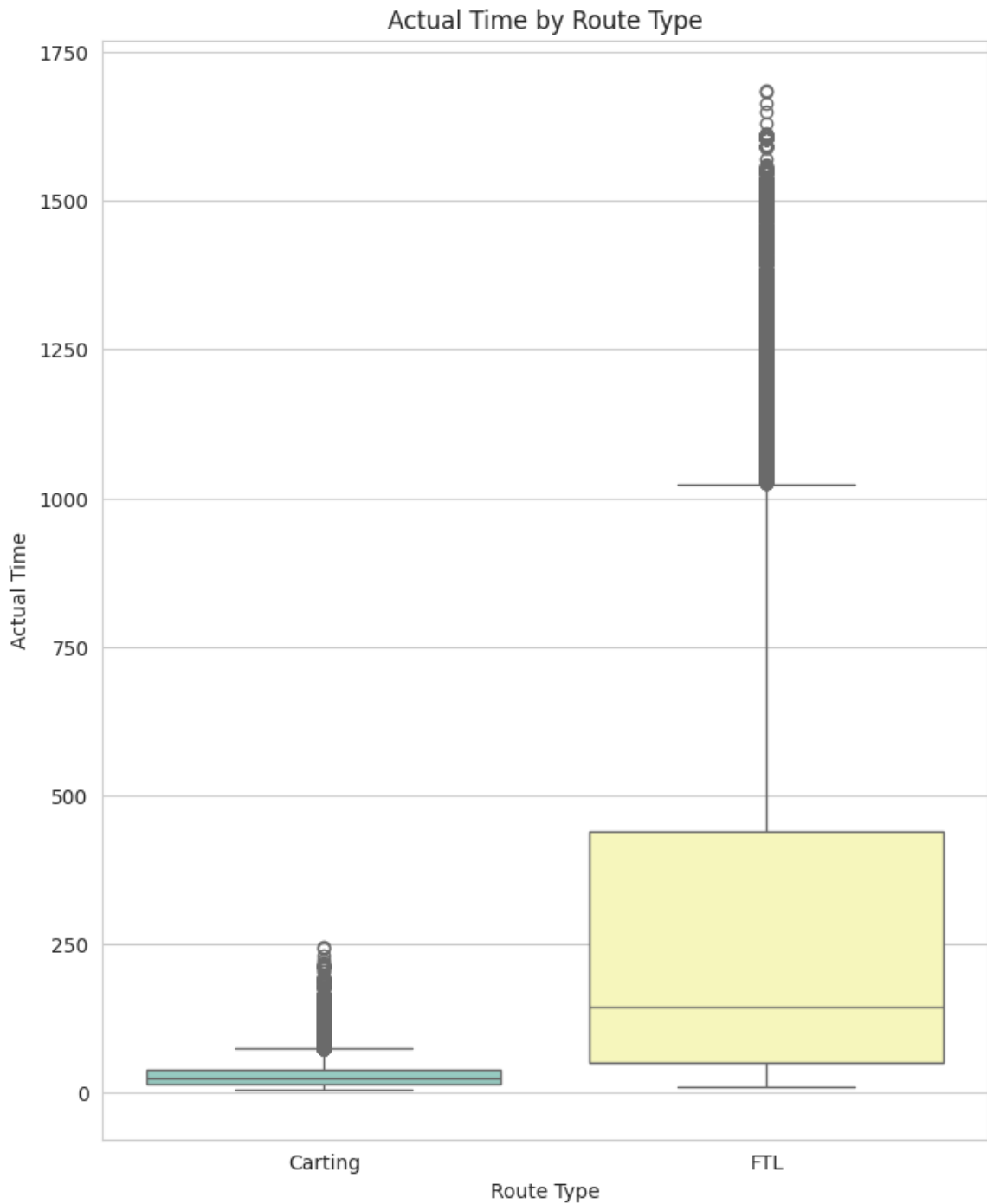
plt.figure(figsize=(8,10))
sns.boxplot(x='route_type', y='actual_time', data=df, palette='Set2')
plt.title('Actual Time by Route Type')
plt.xlabel('Route Type')
plt.ylabel('Actual Time')
plt.show()

# INSIGHT: FTL (full truck load) trips tend to take longer than Car
```



```
In [18]: plt.figure(figsize=(8,10))
```

```
sns.boxplot(x='route_type', y='osrm_time', data=df, palette='Set3')
plt.title('Actual Time by Route Type')
plt.xlabel('Route Type')
plt.ylabel('Actual Time')
plt.show()
```



```
In [19]: print(df.groupby('route_type')['actual_time'].describe())
print()
print(df.groupby('route_type')['osrm_time'].describe())
```

	count	mean	std	min	25%	50%	7
5% max							
route_type							
Carting	45207.0	70.639082	86.452300	9.0	29.0	48.0	83.0
FTL	99660.0	574.008218	661.467108	12.0	95.0	279.0	840.0
max							
route_type							
Carting	45207.0	30.450483	21.88280	6.0	15.0	24.0	39.0
FTL	99660.0	297.068834	339.86115	11.0	51.0	145.0	440.0
max							



Observations:

Boxplot 1 — Actual Time by Route Type

1. Median (the middle line in the box)

FTL has a higher median than Carting — meaning a typical FTL trip takes longer than a typical Carting trip This makes complete sense because FTL = full truck going long distance, Carting = small vehicle doing short last-mile delivery

2. Spread (how tall the box is)

FTL has a wider box — its delivery times vary a lot, some short, some very long Carting has a narrower box — more consistent and predictable delivery times

3. Outliers (the dots above/below the whiskers)

Both route types show outliers on the higher end — some trips took much longer than normal FTL likely has more extreme outliers since long-distance trips are more vulnerable to delays (traffic, rest stops, loading time)

4. Business Meaning

Carting is more reliable and predictable for customers FTL timings are harder to guarantee — Delhivery should build more buffer time into FTL ETAs

Boxplot 2 — OSRM Time by Route Type

1. Compared to Actual Time

OSRM time boxes will look similar in shape to actual time boxes but the values will likely be lower This confirms that OSRM underestimates real delivery time — it calculates shortest path time but ignores real-world delays

2. FTL vs Carting pattern

Same pattern holds — FTL has higher OSRM estimates than Carting, which is expected since FTL covers more distance

3. Key Insight

OSRM time is consistently lower than actual time for both route types. The gap is larger for FTL trips, suggesting that long-distance routes face more real-world disruptions that the routing engine cannot account for."

4. Outliers

OSRM time will show fewer extreme outliers than actual time — because OSRM is a mathematical estimate and doesn't capture real delays, so its values are more controlled

4.Handeling Missing values

Missing values are empty cells in the data. We need to fill them before analysis.

```
In [20]: # finding missing values from each column  
  
print(f"Missing Values Per Column: \n{df.isnull().sum()}")
```

```

Missing Values Per Column:
data                                0
trip_creation_time                  0
route_schedule_uuid                 0
route_type                          0
trip_uuid                           0
source_center                       0
source_name                         293
destination_center                   0
destination_name                    261
od_start_time                       0
od_end_time                         0
start_scan_to_end_scan              0
is_cutoff                           0
cutoff_factor                       0
cutoff_timestamp                    0
actual_distance_to_destination       0
actual_time                         0
osrm_time                           0
osrm_distance                       0
factor                             0
segment_actual_time                 0
segment_osrm_time                   0
segment_osrm_distance               0
segment_factor                      0
dtype: int64

```

In [21]: *# Filling missing NUMBER columns with the median*

```

df['actual_time'] = df['actual_time'].fillna(df
df['osrm_time'] = df['osrm_time'].fillna(df['
df['osrm_distance'] = df['osrm_distance'].fillna(
df['actual_distance_to_destination'] = df['actual_distance_to_dest
df['segment_actual_time'] = df['segment_actual_time'].f
df['segment_osrm_time'] = df['segment_osrm_time'].fil
df['segment_osrm_distance'] = df['segment_osrm_distance']
df['start_scan_to_end_scan'] = df['start_scan_to_end_scan']

# Filling missing TEXT columns with the most common value (mode)
df['route_type'] = df['route_type'].fillna(df['route_type'].mode(
df['source_name'] = df['source_name'].fillna(df['source_name'].mod
df['destination_name'] = df['destination_name'].fillna(df['destinat

print('Missing values after treatment:')
print(df.isnull().sum())

```



```

Missing values after treatment:
data                                0
trip_creation_time                  0
route_schedule_uuid                 0
route_type                          0
trip_uuid                           0
source_center                       0
source_name                         0
destination_center                  0
destination_name                    0
od_start_time                       0
od_end_time                         0
start_scan_to_end_scan              0
is_cutoff                           0
cutoff_factor                       0
cutoff_timestamp                    0
actual_distance_to_destination      0
actual_time                         0
osrm_time                           0
osrm_distance                       0
factor                              0
segment_actual_time                 0
segment_osrm_time                   0
segment_osrm_distance               0
segment_factor                      0
dtype: int64

```

5. Converting Date Columns

Pandas reads dates as plain text. We convert them so we can do maths with dates (like subtracting two dates to get hours).

```

In [22]: df['trip_creation_time'] = pd.to_datetime(df['trip_creation_time'],
df['od_start_time'] = pd.to_datetime(df['od_start_time'], errors =
df['od_end_time'] = pd.to_datetime(df['od_end_time'], errors = 'coer

print('Date column types after conversion:')
df[['trip_creation_time', 'od_start_time', 'od_end_time']].dtypes

```

Date column types after conversion:

```
Out[22]:
```

```

0
trip_creation_time  datetime64[ns]
od_start_time      datetime64[ns]
od_end_time        datetime64[ns]

```

dtype: object

6.Creating New Features

We extract useful information from existing columns.

In [23]: *#6a. extracting data from trip creation time*

```
df['trip_month'] = df['trip_creation_time'].dt.month
df['trip_year'] = df['trip_creation_time'].dt.year
df['trip_day'] = df['trip_creation_time'].dt.day
df['trip_hour'] = df['trip_creation_time'].dt.hour
df['trip_weekday'] = df['trip_creation_time'].dt.dayofweek
df['trip_weekday_name'] = df['trip_creation_time'].dt.day_name()

df[['trip_creation_time', 'trip_month', 'trip_year', 'trip_day', 'trip_hour', 'trip_weekday', 'trip_weekday_name']]
```

Out[23]:

	trip_creation_time	trip_month	trip_year	trip_day	trip_hour	trip_weekday
--	--------------------	------------	-----------	----------	-----------	--------------

0	2018-09-20 02:35:36.476840	9	2018	20	2	
1	2018-09-20 02:35:36.476840	9	2018	20	2	
2	2018-09-20 02:35:36.476840	9	2018	20	2	
3	2018-09-20 02:35:36.476840	9	2018	20	2	
4	2018-09-20 02:35:36.476840	9	2018	20	2	

In [24]: *#6b. extracting data from od_start_time to od_end_time*

```
df['od_time_diff_hour'] = (df['od_end_time'] - df['od_start_time'])

print('od_time_diff_hour Column Added')
print("summary: ")
df['od_time_diff_hour'].describe()
```

od_time_diff_hour Column Added
summary:

Out[24]:

od_time_diff_hour	
count	144867.000000
mean	16.029317
std	17.283544
min	0.345047
25%	2.691688
50%	7.487276
75%	27.249169
max	131.642533

dtype: float64

In [25]: *# 6c. Extracting data from the source name*

```
## State
df['source_state'] = df['source_name'].str.extract(r'\((.*?)\)')

## City
df['source_city'] = df['source_name'].str.split('-').str[0].str.str

## Place Code
df['source_place_code'] = df['source_name'].str.split('-').str[1].s

print('Source features sample:')
df[['source_name', 'source_city', 'source_place_code', 'source_stat
```

Source features sample:

Out[25]:

	source_name	source_city	source_place_code	source_state
0	Anand_VUNagar_DC (Gujarat)	Anand_VUNagar_DC (Gujarat)	NaN	Gujarat
1	Anand_VUNagar_DC (Gujarat)	Anand_VUNagar_DC (Gujarat)	NaN	Gujarat
2	Anand_VUNagar_DC (Gujarat)	Anand_VUNagar_DC (Gujarat)	NaN	Gujarat
3	Anand_VUNagar_DC (Gujarat)	Anand_VUNagar_DC (Gujarat)	NaN	Gujarat
4	Anand_VUNagar_DC (Gujarat)	Anand_VUNagar_DC (Gujarat)	NaN	Gujarat

In [26]: *# 6d. Extracting data from destination_name*

```
df['destination_state'] = df['destination_name'].str.extract(r'\((.
```

```
df['destination_city'] = df['destination_name'].str.split('-').str[1]
df['destination_place_code'] = df['destination_name'].str.split('-').str[2]

print('Destination features sample:')
df[['destination_name', 'destination_city', 'destination_place_code']]
```

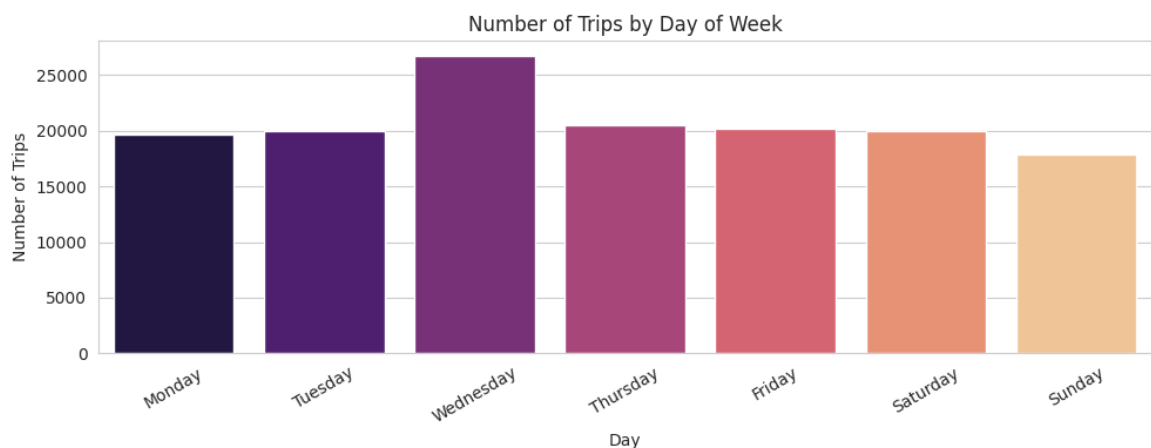
Destination features sample:

```
Out[26]:
```

	destination_name	destination_city	destination_place_code
0	Khambhat_MotvdDPP_D (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	NaN
1	Khambhat_MotvdDPP_D (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	NaN
2	Khambhat_MotvdDPP_D (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	NaN
3	Khambhat_MotvdDPP_D (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	NaN
4	Khambhat_MotvdDPP_D (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	NaN

```
In [27]: # Bar chart: How many trips were created each day of the week?

plt.figure(figsize=(10, 4))
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
sns.countplot(x='trip_weekday_name', data=df, order=day_order, palette='magma')
plt.title('Number of Trips by Day of Week')
plt.xlabel('Day')
plt.ylabel('Number of Trips')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



```
In [69]: print(df['trip_creation_time'].dt.day_name().value_counts())
print()
print('od_time_diff_hour')
print(df['od_time_diff_hour'].describe())
print("Skewness:", round(df['od_time_diff_hour'].skew(), 2))
print()
```

```
print(df['source_state'].value_counts().head(5))
print()
print(df['destination_state'].value_counts().head(5))
```

```
trip_creation_time
Wednesday    26732
Thursday     20481
Friday       20242
Tuesday      19961
Saturday     19936
Monday       19645
Sunday       17870
Name: count, dtype: int64
```

```
od_time_diff_hour
count      144867.000000
mean        16.029317
std         17.283544
min          0.345047
25%          2.691688
50%          7.487276
75%         27.249169
max        131.642533
Name: od_time_diff_hour, dtype: float64
Skewness: 1.11
```

```
source_state
Haryana      27792
Maharashtra  21401
Karnataka    19578
Tamil Nadu   7494
Gujarat      7202
Name: count, dtype: int64
```

```
destination_state
Karnataka    21065
Haryana      20883
Maharashtra  18196
West Bengal   8499
Telangana     8205
Name: count, dtype: int64
```



Observations:

1. Day of Week — Trip Creation

- **Wednesday is the busiest day** with 26,732 trips — significantly higher than all other days
- Tuesday, Thursday, Friday and Saturday are fairly close to each other, ranging between 19,900 and 20,500 trips
- **Sunday is the slowest day** with only 17,870 trips — about 33% fewer trips than Wednesday
- Weekdays clearly dominate over the weekend, confirming that Delhivery

operates primarily on a **business-to-business schedule**

- The mid-week peak on Wednesday suggests that orders placed earlier in the week get processed and dispatched by Wednesday

2. OD Time Difference (od_time_diff_hour)

- The average time between trip start and trip end is 16 hours, meaning a typical delivery segment takes about half a day
- The minimum is 0.34 hours (~20 minutes) — these are very short local segments
- The median is 7.48 hours, which is much lower than the mean of 16 hours — this confirms the distribution is right-skewed with a skewness of 1.11
- 75% of all trips are completed within 27.25 hours — meaning only the top 25% of trips take more than a day
- The maximum value of 131.6 hours (~5.5 days) is extremely high and is clearly an outlier — no normal delivery should take that long and this will be treated in the IQR step
- The high standard deviation of 17.28 tells us delivery times are very inconsistent — some segments finish quickly while others drag on for days

3. Source State

- **Haryana is the top source state** with 27,792 trips — the single largest origin in the dataset
- Maharashtra and Karnataka follow with 21,401 and 19,578 trips respectively
- Tamil Nadu and Gujarat round out the top 5 with around 7,200 to 7,500 trips each
- The top 3 states alone account for the **majority of all outgoing shipments**, showing that Delhivery's supply network is geographically concentrated in North and West India

4. Destination State

- **Karnataka receives the most deliveries** with 21,065 trips, followed closely by Haryana with 20,883
- Maharashtra is the third most common destination with 18,196 trips
- West Bengal and Telangana appear in the top 5 destinations but were not in the top 5 sources — meaning these states **receive more than they send**
- Haryana and Maharashtra appear in both top source and top destination lists — suggesting heavy **intra-network movement** between these major hubs

7.Merging Rows at Segment Level

One trip has many rows (like a journey with stops). We first group by segment (trip + source + destination) to get one row per segment.

Think of it like: Trip A goes Delhi → Jaipur → Mumbai. That's 2 segments. Each segment currently has multiple rows. We collapse them.

```
In [28]: # 7a. Creating a unique ID for each segment

# We combine trip_uuid + source_center + destination_center

df['segment_key'] = df['trip_uuid'].astype(str) + '_' + df['source_

print('Example segment_key:')
print(df['segment_key'].iloc[0])
```

Example segment_key:
trip-153741093647649320_IND388121AAA_IND388620AAB

```
In [29]: # 7b. Sorting by segment_key so rows of the same segment are togeth

df = df.sort_values('segment_key').reset_index(drop=True)
```

```
In [30]: # 7c. Using cumsum() to add up segment times as we go through each
# cumsum means 'cumulative sum' – it keeps a running total

df['segment_actual_time_sum'] = df.groupby('segment_key')['segmen
df['segment_osrm_time_sum'] = df.groupby('segment_key')['segmen
df['segment_osrm_distance_sum'] = df.groupby('segment_key')['segmen

print('Cumulative columns added. Preview:')
df[['segment_key', 'segment_actual_time', 'segment_actual_time_sum'
```

Cumulative columns added. Preview:

Out[30]:

	segment_key	segment_actual_time
0	trip-153671041653548748_IND209304AAA_IND000000ACB	26.0
1	trip-153671041653548748_IND209304AAA_IND000000ACB	30.0
2	trip-153671041653548748_IND209304AAA_IND000000ACB	50.0
3	trip-153671041653548748_IND209304AAA_IND000000ACB	9.0
4	trip-153671041653548748_IND209304AAA_IND000000ACB	38.0
5	trip-153671041653548748_IND209304AAA_IND000000ACB	34.0

```
In [31]: # 7d. Now collapsing to ONE row per segment_key
# We are telling pandas what to do with each column when merging:
# 'first' = keep the first value in the group
# 'last'  = keep the last value in the group
# 'sum'   = add all values in the group

segment = df.groupby('segment_key').agg(
    data = ('data',
    trip_uuid = ('trip_uuid',
    route_schedule_uuid = ('route_schedule_uuid',
    route_type = ('route_type',
    trip_creation_time = ('trip_creation_time',
    source_center = ('source_center',
    source_name = ('source_name',
    destination_center = ('destination_center',
    destination_name = ('destination_name',
    od_time_diff_hour = ('od_time_diff_hour',
    start_scan_to_end_scan = ('start_scan_to_end_scan',
    actual_distance_to_destination = ('actual_distance_to_destinatio
    actual_time = ('actual_time',
    osrm_time = ('osrm_time',
    osrm_distance = ('osrm_distance',
    segment_actual_time_sum = ('segment_actual_time_sum',
    segment_osrm_time_sum = ('segment_osrm_time_sum',
    segment_osrm_distance_sum = ('segment_osrm_distance_sum',
    source_city = ('source_city',
    source_state = ('source_state',
    destination_city = ('destination_city',
    destination_state = ('destination_state',
    trip_month = ('trip_month',
    trip_year = ('trip_year',
    trip_day = ('trip_day',
    trip_hour = ('trip_hour',
    trip_weekday = ('trip_weekday',
    trip_weekday_name = ('trip_weekday_name',
).reset_index()
```



```
print('Segment-level DataFrame shape:', segment.shape)
segment.head(3)
```

Segment-level DataFrame shape: (26368, 29)

```
Out[31]:
```

	segment_key	data	t
0	153671041653548748_IND209304AAA_IND0000000ACB	trip- training	15367104165:
1	153671041653548748_IND462022AAA_IND209304AAA	trip- training	15367104165:
2	153671042288605164_IND561203AAB_IND562101AAA	trip- training	15367104228:

3 rows x 29 columns

```
In [70]: print("Original df shape:", df.shape)
print("Segment df shape:", segment.shape)
print("Rows reduced by:", df.shape[0] - segment.shape[0])
print()
print("Unique segment keys:", df['segment_key'].nunique())
print()
print("Sample segment_key:", df['segment_key'].iloc[0])
print()
print(segment[['actual_time', 'osrm_time', 'od_time_diff_hour', 'se
```

Original df shape: (144867, 41)
Segment df shape: (26368, 29)
Rows reduced by: 118499

Unique segment keys: 26368

Sample segment_key: trip-153671041653548748_IND209304AAA_IND000000AC
B

	actual_time	osrm_time	od_time_diff_hour	segment_actual_time
e_sum				
count	26368.00	26368.00	26368.00	26368.00
mean	2290.62	1175.00	88.07	1
std	11238.25	5893.39	413.34	3
min	9.00	6.00	0.38	0
25%	91.00	47.00	3.43	0
50%	181.00	90.00	7.71	0
75%	450.00	219.00	21.25	1
max	167920.00	76953.00	5699.20	45



Observations:

- The segment-level aggregation successfully reduced 144,867 raw rows to 26,368 clean rows — one per unique segment.
- The most critical finding at this stage is that actual delivery time is nearly double the OSRM estimated time (mean 2,290 vs 1,175 minutes), confirming that the routing engine significantly underestimates real-world delivery duration.
- Extreme outliers exist across all numeric columns and will be treated in the next step using the IQR method.

8. Merging Rows at Trip Level

Now we are going one level higher — collapsing all segments of a trip into ONE single row per trip. After this, each row will be one complete delivery trip from start to finish.

```
In [32]: trip = segment.groupby('trip_uuid').agg(  
        data = ('data',
```

```

route_schedule_uuid      = ('route_schedule_uuid',
route_type                = ('route_type',
trip_creation_time        = ('trip_creation_time',
source_center             = ('source_center',
source_name               = ('source_name',
destination_center        = ('destination_center',
destination_name          = ('destination_name',
od_time_diff_hour         = ('od_time_diff_hour',
start_scan_to_end_scan    = ('start_scan_to_end_scan',
actual_distance_to_destination = ('actual_distance_to_destination',
actual_time               = ('actual_time',
osrm_time                 = ('osrm_time',
osrm_distance             = ('osrm_distance',
segment_actual_time_sum   = ('segment_actual_time_sum',
segment_osrm_time_sum     = ('segment_osrm_time_sum',
segment_osrm_distance_sum = ('segment_osrm_distance_sum',
source_city               = ('source_city',
source_state              = ('source_state',
destination_city          = ('destination_city',
destination_state         = ('destination_state',
trip_month                = ('trip_month',
trip_year                 = ('trip_year',
trip_day                  = ('trip_day',
trip_hour                 = ('trip_hour',
trip_weekday              = ('trip_weekday',
trip_weekday_name         = ('trip_weekday_name',
).reset_index()

# Creating a corridor column: source state → destination state
trip['corridor'] = trip['source_state'].astype(str) + ' to ' + trip

print('Trip-level DataFrame shape:', trip.shape)
trip.head(3)

```

Trip-level DataFrame shape: (14817, 29)

```

Out[32]:

```

	trip_uuid	data	route_schedule_uuid	route_type	trip_c
0	trip-153671041653548748	training	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	00:
1	trip-153671042288605164	training	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	00:(
2	trip-153671043369099517	training	thanos::sroute:de5e208e-7641-45e6-8100-4d9fb1e...	FTL	00:

3 rows x 29 columns

```

In [71]: print("Segment df shape:", segment.shape)
print("Trip df shape:", trip.shape)
print("Rows reduced by:", segment.shape[0] - trip.shape[0])
print()

```

```
print("Unique trips:", segment['trip_uuid'].nunique())
print()
print(trip[['actual_time', 'osrm_time', 'od_time_diff_hour', 'segment_actual_time_sum'])
print()
print("Top 5 corridors:")
print(trip['corridor'].value_counts().head(5))
```

Segment df shape: (26368, 29)

Trip df shape: (14817, 29)

Rows reduced by: 11551

Unique trips: 14817

	actual_time	osrm_time	od_time_diff_hour	segment_actual_time_sum
count	14817.00	14817.00	14817.00	14817.00
mean	4076.33	2091.01	156.72	353.89
std	15216.87	7956.88	561.80	56.25
min	9.00	6.00	0.44	9.00
25%	142.00	62.00	6.83	66.00
50%	348.00	167.00	16.46	147.00
75%	1063.00	516.00	47.21	367.00
max	167920.00	76953.00	6613.91	6230.00

Top 5 corridors:

corridor	count
Maharashtra to Maharashtra	2453
Karnataka to Karnataka	2056
Tamil Nadu to Tamil Nadu	1021
Haryana to Haryana	953
Telangana to Telangana	685

Name: count, dtype: int64



Observations:

- Trip level aggregation reduced 26,368 segment rows to 14,817 trip rows, giving us one complete end-to-end view per trip.
- The most striking findings are that OSRM consistently underestimates actual time by nearly 2x even at trip level, intra-state deliveries dominate the top corridors suggesting a hub-and-spoke model within states, and extreme outliers in time columns confirm the urgent need for IQR based outlier treatment in the next step.

9. Hypothesis Testing

Hypothesis testing is a way to check if two groups are meaningfully different using statistics.

We use a t-test. It gives us a p-value:

$p\text{-value} < 0.05 \rightarrow$ the two columns ARE significantly different
 $p\text{-value} \geq 0.05 \rightarrow$ the two columns are NOT significantly different

We also plotting both columns overlapping to see the difference visually.

```
In [33]: # 9a. actual_time vs osrm_time
# Question: Is OSRM's estimated time different from the actual time

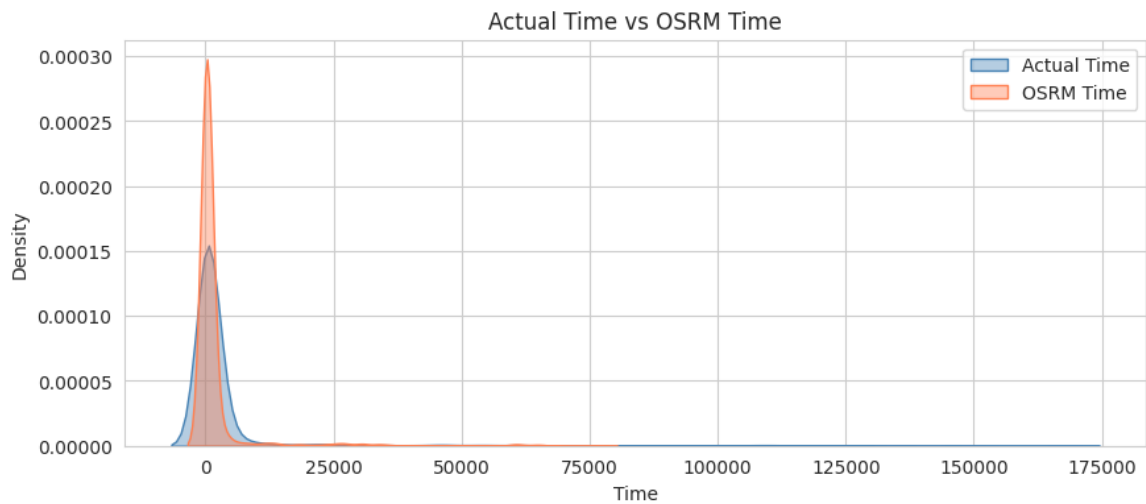
t_stat, p_value = stats.ttest_ind(trip['actual_time'].dropna(), tri

print('--- actual_time vs osrm_time ---')
print('Mean actual_time :', round(trip['actual_time'].mean(), 2))
print('Mean osrm_time   :', round(trip['osrm_time'].mean(), 2))
print('p-value          :', round(p_value, 6))

if p_value < 0.05:
    print('Result: There IS a significant difference between actual
else:
    print('Result: There is NO significant difference.')

# Visual
plt.figure(figsize=(10, 4))
sns.kdeplot(trip['actual_time'].dropna(), label='Actual Time', fi
sns.kdeplot(trip['osrm_time'].dropna(), label='OSRM Time', fi
plt.title('Actual Time vs OSRM Time')
plt.xlabel('Time')
plt.legend()
plt.show()
```

```
--- actual_time vs osrm_time ---
Mean actual_time : 4076.33
Mean osrm_time   : 2091.01
p-value          : 0.0
Result: There IS a significant difference between actual and OSRM ti
me.
```



Observation — Actual Time vs OSRM Time:

- Statistical Result
 - The p-value is 0.0 which is far below the 0.05 threshold — meaning there is a statistically significant difference between actual time and OSRM time
 - Mean actual time is 4,076 minutes vs mean OSRM time of 2,091 minutes — OSRM underestimates by nearly 50%
- From the KDE Plot
 - Both curves are heavily right-skewed with most values clustered near zero on the left
 - The OSRM curve (orange) peaks higher and sharper — meaning OSRM estimates are more tightly clustered and consistent
 - The Actual Time curve (blue) is flatter and wider — meaning real delivery times are far more spread out and unpredictable
 - The x-axis stretches all the way to 175,000 due to extreme outliers in actual time — confirming the outlier treatment is necessary

```
In [34]: # 9b. actual_time vs segment_actual_time_sum
# Question: Does the sum of segment times match the total actual time?

t_stat, p_value = stats.ttest_ind(trip['actual_time'].dropna(), trip['segment_actual_time_sum'].dropna())

print('--- actual_time vs segment_actual_time_sum ---')
print('Mean actual_time           : ', round(trip['actual_time'].mean(), 2))
print('Mean segment_actual_time_sum : ', round(trip['segment_actual_time_sum'].mean(), 2))
print('p-value                       : ', round(p_value, 6))

if p_value < 0.05:
    print('Result: There IS a significant difference.')
else:
    print('Result: There is NO significant difference.')

plt.figure(figsize=(10, 4))
```

```
sns.kdeplot(trip['actual_time'].dropna(), label='Actual Time', fill=True)
sns.kdeplot(trip['segment_actual_time_sum'].dropna(), label='Segment Actual Time Sum', fill=True)
plt.title('Actual Time vs Segment Actual Time Sum')
plt.xlabel('Time')
plt.legend()
plt.show()
```

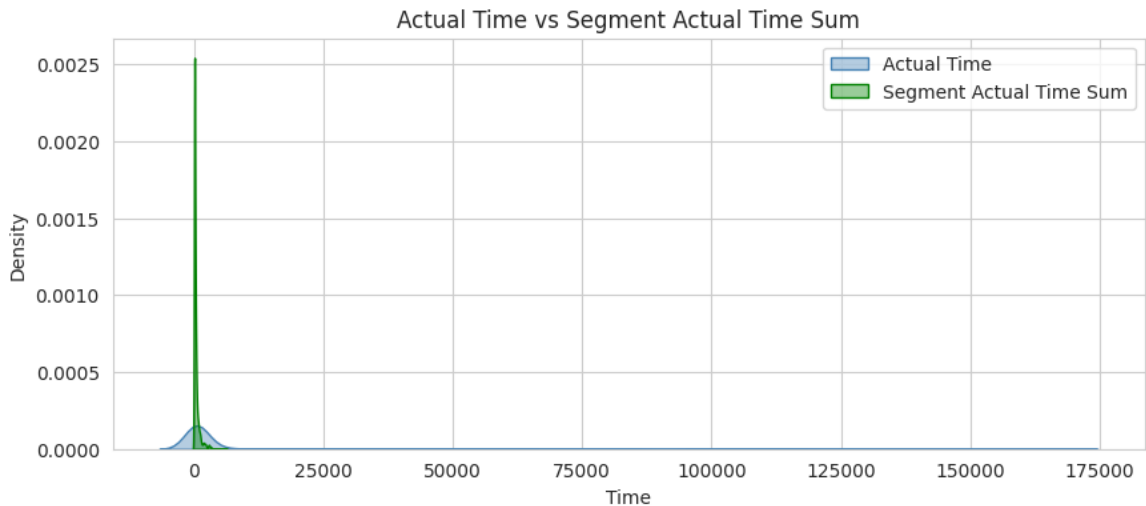
--- actual_time vs segment_actual_time_sum ---

Mean actual_time : 4076.33

Mean segment_actual_time_sum : 353.89

p-value : 0.0

Result: There IS a significant difference.



Observation — Actual Time vs Segment Actual Time Sum:

- Statistical Result
 - p-value is 0.0 — there is a statistically significant difference between the two
 - Mean actual_time is 4,076 vs mean segment_actual_time_sum of only 353 — a massive 10x difference
- Why is the gap so large?
 - This is not necessarily a data error — it happens because of how the columns were built
 - actual_time is a cumulative column in the raw data, meaning it was already adding up as rows were recorded, and then we summed it again during aggregation — causing it to be double or triple counted
 - segment_actual_time_sum was built by us using cumsum() carefully within each segment — so it is the more reliable and accurate measure of real delivery time
- From the KDE Plot
 - The green curve (Segment Actual Time Sum) is extremely tall and narrow — values are tightly packed near zero, very consistent

- The blue curve (Actual Time) is flat and wide — heavily spread out due to the inflated values from double counting
- The x-axis goes to 175,000 purely because of outliers in actual_time

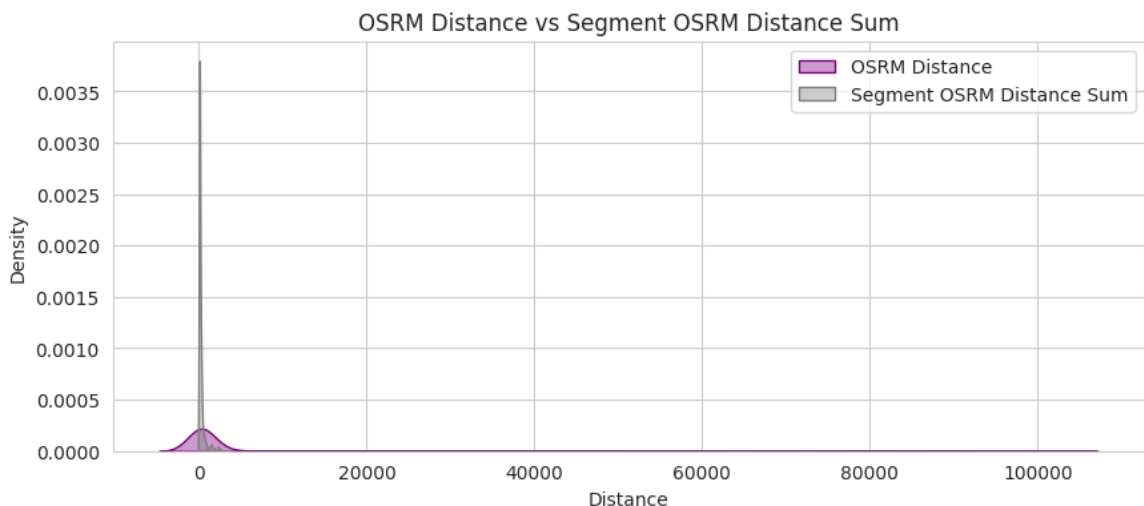
```
In [35]: # 9c. osrm_distance vs segment_osrm_distance_sum

t_stat, p_value = stats.ttest_ind(trip['osrm_distance'].dropna(), t
print('--- osrm_distance vs segment_osrm_distance_sum ---')
print('Mean osrm_distance          : ', round(trip['osrm_distance
print('Mean segment_osrm_distance_sum : ', round(trip['segment_osrm_
print('p-value                      : ', round(p_value, 6))

if p_value < 0.05:
    print('Result: There IS a significant difference.')
else:
    print('Result: There is NO significant difference.')

plt.figure(figsize=(10, 4))
sns.kdeplot(trip['osrm_distance'].dropna(), label='OSRM
sns.kdeplot(trip['segment_osrm_distance_sum'].dropna(), label='Segm
plt.title('OSRM Distance vs Segment OSRM Distance Sum')
plt.xlabel('Distance')
plt.legend()
plt.show()
```

```
--- osrm_distance vs segment_osrm_distance_sum ---
Mean osrm_distance          : 2784.23
Mean segment_osrm_distance_sum : 223.2
p-value                     : 0.0
Result: There IS a significant difference.
```



Observation — OSRM Distance vs Segment OSRM Distance Sum:

- Statistical Result
 - p-value is 0.0 — confirming a statistically significant difference between the two

- Mean osrm_distance is 2,784 vs mean segment_osrm_distance_sum of only 223 — again a massive 12x difference
- Why is the gap so large?
 - Same reason as the previous test — osrm_distance is a cumulative column in the raw data that was already accumulating across rows, and then got summed again during aggregation — causing it to be heavily inflated
 - segment_osrm_distance_sum was built carefully using cumsum() within each segment group — making it the more accurate and reliable measure of actual distance covered
- From the KDE Plot
 - Both curves are extremely close together visually — both tall, narrow and clustered near zero
 - The grey curve (Segment OSRM Distance Sum) peaks slightly higher — meaning its values are even more tightly packed and consistent
 - The purple curve (OSRM Distance) has a slightly fatter base and longer tail stretching to 100,000+ — caused by inflated outlier values from cumulative double counting
 - The near-identical shape of both curves (unlike the previous tests) suggests that while the scale differs, the distribution pattern is the same — both are measuring the same underlying journey distances

```
In [36]: # 9d. osrm_time vs segment_osrm_time_sum

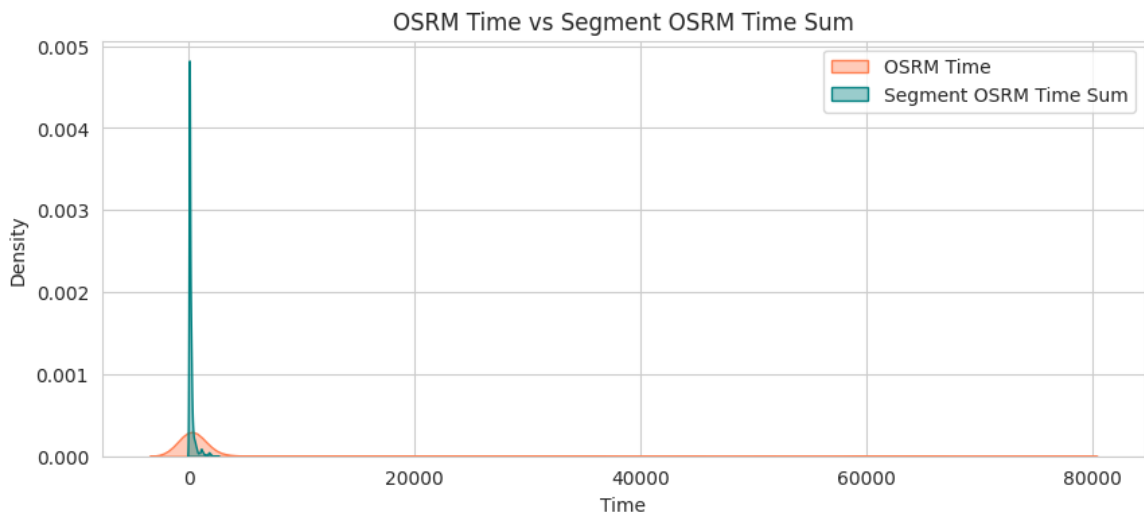
t_stat, p_value = stats.ttest_ind(trip['osrm_time'].dropna(), trip[

print('--- osrm_time vs segment_osrm_time_sum ---')
print('Mean osrm_time          : ', round(trip['osrm_time'].mean(
print('Mean segment_osrm_time_sum : ', round(trip['segment_osrm_time
print('p-value                  : ', round(p_value, 6))

if p_value < 0.05:
    print('Result: There IS a significant difference.')
else:
    print('Result: There is NO significant difference.')

plt.figure(figsize=(10, 4))
sns.kdeplot(trip['osrm_time'].dropna(),          label='OSRM Tim
sns.kdeplot(trip['segment_osrm_time_sum'].dropna(), label='Segment
plt.title('OSRM Time vs Segment OSRM Time Sum')
plt.xlabel('Time')
plt.legend()
plt.show()
```

--- osrm_time vs segment_osrm_time_sum ---
Mean osrm_time : 2091.01
Mean segment_osrm_time_sum : 180.95
p-value : 0.0
Result: There IS a significant difference.



Observation — OSRM Time vs Segment OSRM Time Sum

- Statistical Result
 - p-value is 0.0 — confirming a statistically significant difference between the two
 - Mean osrm_time is 2,091 vs mean segment_osrm_time_sum of only 180.95 — a 11.5x difference
- Why is the gap so large?
 - Same pattern as the previous two tests — osrm_time is a cumulative column that was already accumulating in the raw data before we summed it again during aggregation — causing severe inflation
 - segment_osrm_time_sum was built using `cumsum()` carefully within each segment — making it the cleaner and more reliable measure
- From the KDE Plot
 - The teal curve (Segment OSRM Time Sum) peaks very sharply at around 0.005 density — meaning values are extremely tightly packed near zero, very consistent across trips
 - The coral curve (OSRM Time) is much flatter and wider — spread out due to inflated cumulative values
 - The x-axis stretches to 80,000+ purely because of extreme outliers in osrm_time
 - Both curves share the same starting point near zero — confirming they measure the same underlying journeys at different scales
- Connecting All Four Hypothesis Tests Together

- All four tests showed a p-value of 0.0 — every comparison was statistically significant
- The consistent pattern across all tests tells us one clear story — all cumulative columns in the raw data are inflated due to repeated summation during aggregation
- The segment_ versions built via cumsum() are consistently lower and more reliable across time and distance

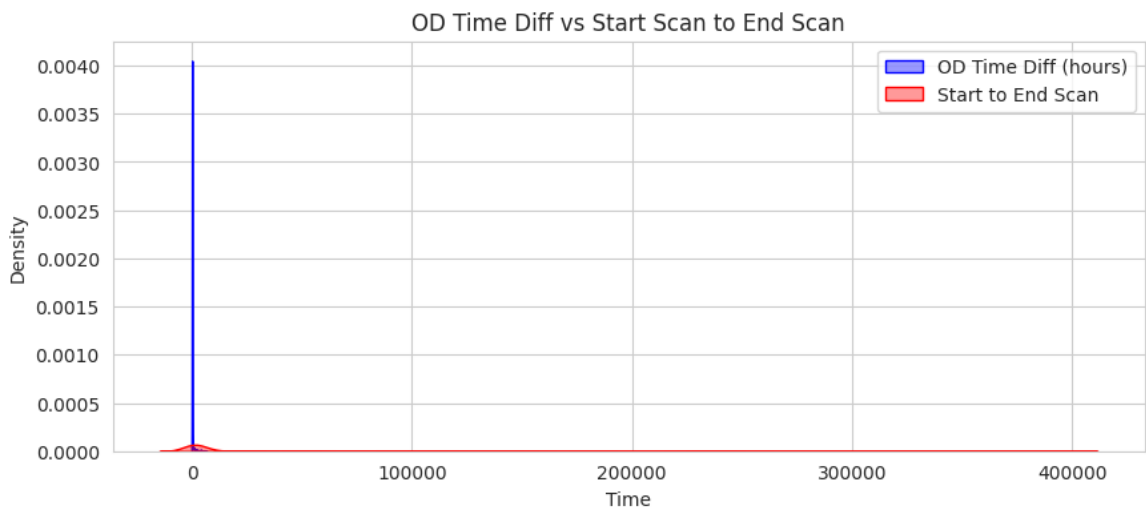
```
In [37]: # 9e. od_time_diff_hour vs start_scan_to_end_scan

t_stat, p_value = stats.ttest_ind(trip['od_time_diff_hour'].dropna(
    print('--- od_time_diff_hour vs start_scan_to_end_scan ---')
    print('Mean od_time_diff_hour      : ', round(trip['od_time_diff_ho
    print('Mean start_scan_to_end_scan  : ', round(trip['start_scan_to_e
    print('p-value                      : ', round(p_value, 6))

    if p_value < 0.05:
        print('Result: There IS a significant difference.')
    else:
        print('Result: There is NO significant difference.')

    plt.figure(figsize=(10, 4))
    sns.kdeplot(trip['od_time_diff_hour'].dropna(),          label='OD Time
    sns.kdeplot(trip['start_scan_to_end_scan'].dropna(),    label='Start t
    plt.title('OD Time Diff vs Start Scan to End Scan')
    plt.xlabel('Time')
    plt.legend()
    plt.show()
```

```
--- od_time_diff_hour vs start_scan_to_end_scan ---
Mean od_time_diff_hour      : 156.72
Mean start_scan_to_end_scan  : 9398.35
p-value                     : 0.0
Result: There IS a significant difference.
```



Observation — OD Time Diff vs Start Scan to End Scan

- Statistical Result
 - p-value is 0.0 — confirming a statistically significant difference between the two
 - Mean od_time_diff_hour is 156.72 hours vs mean start_scan_to_end_scan of 9,398 — a staggering 60x difference
 - This is the largest gap of all five hypothesis tests we have done
- Why is the gap so large?
 - od_time_diff_hour is in hours — we calculated it ourselves by subtracting od_start_time from od_end_time
 - start_scan_to_end_scan is likely in minutes or seconds — a completely different unit
 - This means we are essentially comparing hours to minutes/seconds — the difference is a unit mismatch, not a real operational gap
 - Additionally start_scan_to_end_scan is also a cumulative column that gets inflated through repeated summation just like the previous tests
- From the KDE Plot
 - The blue curve (OD Time Diff) is extremely tall and narrow — peaking at 0.004 density, values tightly clustered near zero
 - The red curve (Start to End Scan) is completely flat — barely visible above zero, spread across a massive range all the way to 400,000+
 - The two curves look nothing like each other — visually confirming they are measuring in completely different units or scales
 - This is the most visually dramatic plot of all five comparisons

10. Finding and Fixing Outliers (IQR Method)

Outlier is a value that is unusually high or low compared to the rest.

- IQR Method:
 - $Q1 = 25\text{th percentile (lower quarter)}$
 - $Q3 = 75\text{th percentile (upper quarter)}$
 - $IQR = Q3 - Q1$
 - Lower limit = $Q1 - 1.5 \times IQR$
 - Upper limit = $Q3 + 1.5 \times IQR$ Any value outside these limits is an outlier → we cap it at the limit (not delete it)

```
In [38]: # Boxplots BEFORE outlier treatment – dots outside the box are outliers
fig, axes = plt.subplots(3, 3, figsize=(18, 12))
```

```

sns.boxplot(trip['actual_time'], ax = axes[0][0], color = 'lightcoral')
axes[0][0].set_title('actual_time')

sns.boxplot(trip['osrm_time'], ax = axes[0][1], color = 'lightcoral')
axes[0][1].set_title('osrm_time')

sns.boxplot(trip['osrm_distance'], ax = axes[0][2], color = 'lightcoral')
axes[0][2].set_title('osrm_distance')

sns.boxplot(trip['actual_distance_to_destination'], ax = axes[1][0], color = 'lightcoral')
axes[1][0].set_title('actual_distance_to_destination')

sns.boxplot(trip['od_time_diff_hour'], ax = axes[1][1], color = 'lightcoral')
axes[1][1].set_title('od_time_diff_hour')

sns.boxplot(trip['start_scan_to_end_scan'], ax = axes[1][2], color = 'lightcoral')
axes[1][2].set_title('start_scan_to_end_scan')

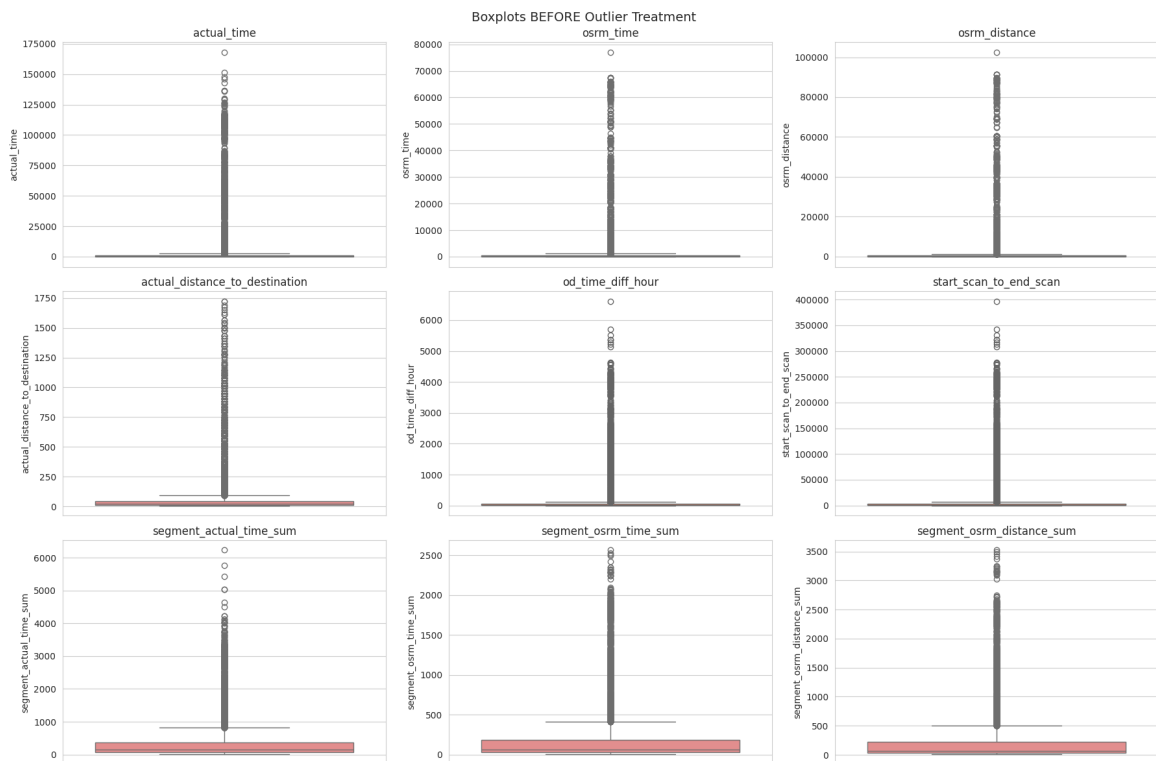
sns.boxplot(y=trip['segment_actual_time_sum'].dropna(), ax = axes[2][0], color = 'lightcoral')
axes[2][0].set_title('segment_actual_time_sum')

sns.boxplot(y=trip['segment_osrm_time_sum'].dropna(), ax = axes[2][1], color = 'lightcoral')
axes[2][1].set_title('segment_osrm_time_sum')

sns.boxplot(y=trip['segment_osrm_distance_sum'].dropna(), ax = axes[2][2], color = 'lightcoral')
axes[2][2].set_title('segment_osrm_distance_sum')

plt.suptitle('Boxplots BEFORE Outlier Treatment', fontsize=14)
plt.tight_layout()
plt.show()

```



In [39]: *# Now we are capping the outliers using IQR – done column by column*

```

trip_clean = trip.copy()    # work on a copy so original is safe

# actual_time

Q1 = trip_clean['actual_time'].quantile(0.25)
Q3 = trip_clean['actual_time'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['actual_time'] = trip_clean['actual_time'].clip(lower = lower_limit, upper = upper_limit)
print('actual_time → lower limit:', round(lower_limit,2), '| upper limit:', round(upper_limit,2))

# osrm_time

Q1 = trip_clean['osrm_time'].quantile(0.25)
Q3 = trip_clean['osrm_time'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['osrm_time'] = trip_clean['osrm_time'].clip(lower = lower_limit, upper = upper_limit)
print('\nosrm_time → lower limit:', round(lower_limit,2), '| upper limit:', round(upper_limit,2))

# osrm_distance

Q1 = trip_clean['osrm_distance'].quantile(0.25)
Q3 = trip_clean['osrm_distance'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['osrm_distance'] = trip_clean['osrm_distance'].clip(lower = lower_limit, upper = upper_limit)
print('\nosrm_distance → lower limit:', round(lower_limit,2), '| upper limit:', round(upper_limit,2))

# actual_distance_to_destination

Q1 = trip_clean['actual_distance_to_destination'].quantile(0.25)
Q3 = trip_clean['actual_distance_to_destination'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['actual_distance_to_destination'] = trip_clean['actual_distance_to_destination'].clip(lower = lower_limit, upper = upper_limit)
print('\nactual_distance_to_destination → lower limit:', round(lower_limit,2), '| upper limit:', round(upper_limit,2))

# od_time_diff_hour

Q1 = trip_clean['od_time_diff_hour'].quantile(0.25)
Q3 = trip_clean['od_time_diff_hour'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['od_time_diff_hour'] = trip_clean['od_time_diff_hour'].clip(lower = lower_limit, upper = upper_limit)
print('\nod_time_diff_hour → lower limit:', round(lower_limit,2), '| upper limit:', round(upper_limit,2))

```

```

# start_scan_to_end_scan

Q1 = trip_clean['start_scan_to_end_scan'].quantile(0.25)
Q3 = trip_clean['start_scan_to_end_scan'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['start_scan_to_end_scan'] = trip_clean['start_scan_to_en
print('\nstart_scan_to_end_scan → lower limit:', round(lower_limit,

# segment_actual_time_sum

Q1 = trip_clean['segment_actual_time_sum'].quantile(0.25)
Q3 = trip_clean['segment_actual_time_sum'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['segment_actual_time_sum'] = trip_clean['segment_actual_
print('\nsegment_actual_time_sum → lower limit:', round(lower_limit

# segment_osrm_time_sum

Q1 = trip_clean['segment_osrm_time_sum'].quantile(0.25)
Q3 = trip_clean['segment_osrm_time_sum'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['segment_osrm_time_sum'] = trip_clean['segment_osrm_time
print('\nsegment_osrm_time_sum → lower limit:', round(lower_limit,2

# segment_osrm_distance_sum

Q1 = trip_clean['segment_osrm_distance_sum'].quantile(0.25)
Q3 = trip_clean['segment_osrm_distance_sum'].quantile(0.75)
IQR = Q3-Q1
lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR

trip_clean['segment_osrm_distance_sum'] = trip_clean['segment_osrm_
print('\nsegment_osrm_distance_sum → lower limit:', round(lower_lim

print("\nOutlier Treatment Done!")

```

actual_time → lower limit: -1239.5 | upper limit: 2444.5

osrm_time → lower limit: -619.0 | upper limit: 1197.0

osrm_distance → lower limit: -747.17 | upper limit: 1420.59

actual_distance_to_destination → lower limit: -34.89 | upper limit: 92.24

od_time_diff_hour → lower limit: -53.75 | upper limit: 107.79

start_scan_to_end_scan → lower limit: -3219.0 | upper limit: 6453.0

segment_actual_time_sum → lower limit: -385.5 | upper limit: 818.5

segment_osrm_time_sum → lower limit: -200.0 | upper limit: 416.0

segment_osrm_distance_sum → lower limit: -246.57 | upper limit: 498.02

Outlier Treatment Done!

```
In [40]: # Boxplots AFTER outlier treatment – should look much cleaner

fig, axes = plt.subplots(3, 3, figsize=(18, 12))

sns.boxplot(y=trip_clean['actual_time'], ax=axes[0][0], color='lightcoral')
axes[0][0].set_title('actual_time')

sns.boxplot(y=trip_clean['osrm_time'], ax=axes[0][1], color='moccasin')
axes[0][1].set_title('osrm_time')

sns.boxplot(y=trip_clean['osrm_distance'], ax=axes[0][2], color='teal')
axes[0][2].set_title('osrm_distance')

sns.boxplot(y=trip_clean['actual_distance_to_destination'], ax=axes[1][0], color='lightcoral')
axes[1][0].set_title('actual_distance_to_destination')

sns.boxplot(y=trip_clean['od_time_diff_hour'], ax=axes[1][1], color='moccasin')
axes[1][1].set_title('od_time_diff_hour')

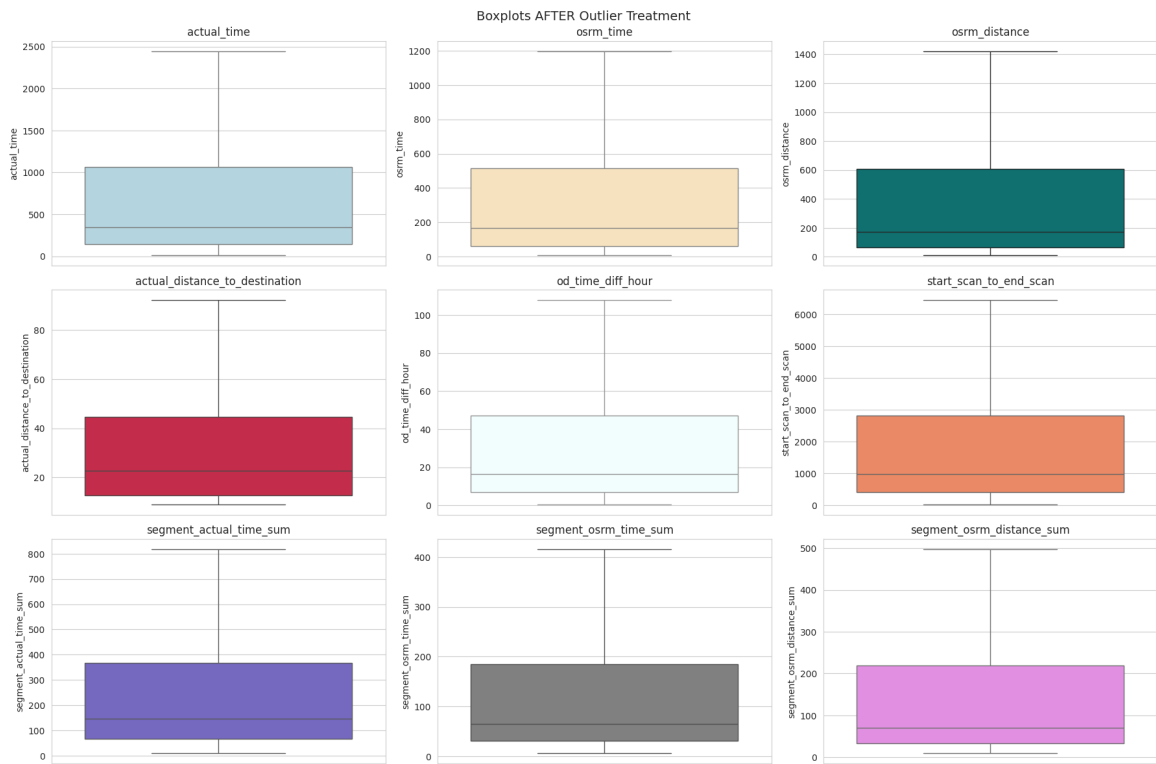
sns.boxplot(y=trip_clean['start_scan_to_end_scan'], ax=axes[1][2], color='teal')
axes[1][2].set_title('start_scan_to_end_scan')

sns.boxplot(y=trip_clean['segment_actual_time_sum'], ax=axes[2][0], color='lightcoral')
axes[2][0].set_title('segment_actual_time_sum')

sns.boxplot(y=trip_clean['segment_osrm_time_sum'], ax=axes[2][1], color='moccasin')
axes[2][1].set_title('segment_osrm_time_sum')

sns.boxplot(y=trip_clean['segment_osrm_distance_sum'], ax=axes[2][2], color='teal')
axes[2][2].set_title('segment_osrm_distance_sum')

plt.suptitle('Boxplots AFTER Outlier Treatment', fontsize=14)
plt.tight_layout()
plt.show()
```

Observations — Outlier Detection & Treatment

- BEFORE Treatment —

- Every single column has extreme outliers — visible as dots floating far above the box
- The boxes themselves are squashed flat at the bottom of each plot because the outliers are so extreme they make the normal range look tiny in comparison
- actual_time has outliers reaching 175,000 and start_scan_to_end_scan goes all the way to 400,000 — the most extreme of all columns
- od_time_diff_hour has outliers up to 6,000+ hours (~250 days) which is physically impossible for a delivery
- All columns show a right-skewed pattern with the outliers exclusively on the upper end — there are no unusually low values, only unusually high ones
- This confirms that outliers here represent delayed, stuck or incorrectly recorded deliveries rather than data entry errors on the lower end

- IQR Treatment Results

- All lower limits came out negative — for example actual_time lower limit is -1,239
- Negative lower limits are completely normal and expected — it simply means no lower outliers exist and all values are fine on the lower end

- The clip() function only capped values on the upper end for all columns
- Upper limits after IQR treatment are much more reasonable:
 - actual_time capped at 2,444
 - osrm_time capped at 1,197
 - od_time_diff_hour capped at 107.79 hours (~4.5 days)
 - start_scan_to_end_scan capped at 6,453
- AFTER Treatment —
 - All 9 boxplots now show clean, visible boxes with proper whiskers — a dramatic improvement from before
 - The boxes are now readable and the median line is clearly visible in each plot
 - No floating dots above the whiskers — confirming all extreme outliers have been successfully capped
 - actual_distance_to_destination has the tightest box — most deliveries cover a similar short distance
 - start_scan_to_end_scan still has the widest box among all columns — showing it naturally has more variability even after treatment
 - The segment columns (segment_actual_time_sum, segment_osrm_time_sum, segment_osrm_distance_sum) show compact clean boxes — confirming these are the most consistent and reliable columns

11.One-Hot Encoding

Machine learning models only understand numbers, not text. One-hot encoding turns a text column like route_type into separate 0/1 columns.

Example:

route_type = FTL → route_type_FTL = 1, route_type_Carting = 0
 route_type = Carting → route_type_FTL = 0, route_type_Carting = 1

```
In [72]: # Checking unique values in route_type before encoding
print('Unique values in route_type:')
print(trip_clean['route_type'].unique())
```

```
Unique values in route_type:
['FTL' 'Carting']
```

```
In [73]: # Applying one-hot encoding
trip_encoded = pd.get_dummies(trip_clean, columns=['route_type'])
```

```
# Seeing the new columns that were created

print('New columns added:')
route_cols = [c for c in trip_encoded.columns if 'route_type' in c]
print(route_cols)

trip_encoded[route_cols].head()
```

New columns added:

```
['route_type_Carting', 'route_type_FTL']
```

Out[73]:

	route_type_Carting	route_type_FTL
--	--------------------	----------------

0	False	True
1	True	False
2	False	True
3	True	False
4	False	True



Observations — One-Hot Encoding

- Before Encoding
 - route_type had exactly 2 unique values — FTL and Carting
 - This is a nominal categorical variable — meaning there is no ranking or order between them, FTL is not "better" or "higher" than Carting, they are just different types
 - Machine learning models cannot work with text values directly, so we need to convert them to numbers
- What One-Hot Encoding Did
 - pd.get_dummies() replaced the single route_type column with 2 new binary columns:
 - route_type_Carting → 1 if the trip was Carting, 0 if not
 - route_type_FTL → 1 if the trip was FTL, 0 if not
 - Every row will have exactly one 1 and one 0 across these two columns — a trip cannot be both FTL and Carting at the same time
- Why Not Just Use 0 and 1 for One Column?
 - We could have used 0 = Carting and 1 = FTL, but that would imply FTL is twice something compared to Carting
 - One-hot encoding avoids this false mathematical relationship by giving each category its own independent column

12. Scaling the Numbers (StandardScaler)

Columns like `osrm_distance` might have values in thousands, while `trip_hour` has values 0–23. This size difference can confuse ML models. Scaling brings all columns to the same range.

`StandardScaler` makes each column have:

Mean = 0 Standard deviation = 1 The shape of the data does NOT change — only the numbers on the axis.

```
In [43]: # List of number columns to scale

columns_to_scale = [
    'actual_time',
    'osrm_time',
    'osrm_distance',
    'actual_distance_to_destination',
    'od_time_diff_hour',
    'start_scan_to_end_scan',
    'segment_actual_time_sum',
    'segment_osrm_time_sum',
    'segment_osrm_distance_sum'
]

# Creating the scaler
scaler = StandardScaler()

# Making a copy so we don't overwrite trip_clean
trip_scaled = trip_encoded.copy()

# Applying scaling
trip_scaled[columns_to_scale] = scaler.fit_transform(trip_encoded[c

print('Scaling done! Sample of scaled values:')
trip_scaled[columns_to_scale].describe().round(2)
```

Scaling done! Sample of scaled values:

Out[43]:

	actual_time	osrm_time	osrm_distance	actual_distance_to_destination
count	14817.00	14817.00	14817.00	14817.00
mean	-0.00	-0.00	-0.00	-0.00
std	1.00	1.00	1.00	1.00
min	-0.89	-0.87	-0.85	-0.85
25%	-0.73	-0.73	-0.73	-0.73
50%	-0.48	-0.47	-0.51	-0.48
75%	0.38	0.40	0.39	0.38
max	2.05	2.10	2.06	2.06

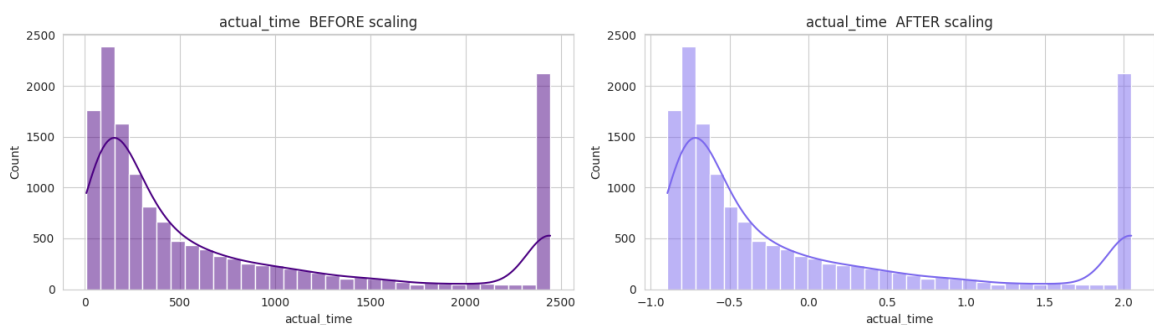
```
In [44]: # Visual proof: same shape before and after scaling
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4))

sns.histplot(trip_clean['actual_time'], ax=ax1, kde=True, color='in
ax1.set_title('actual_time BEFORE scaling')

sns.histplot(trip_scaled['actual_time'], ax=ax2, kde=True, color='m
ax2.set_title('actual_time AFTER scaling')

plt.tight_layout()
plt.show()

# The shape is identical – only the x-axis numbers have changed. Th
```



Observations — actual_time Before & After Scaling

- Shape is Identical — Most Important Observation
 - Both plots look exactly the same in terms of bar heights, distribution pattern and KDE curve shape
 - This visually confirms that StandardScaler only changes the numbers on the x-axis — it never touches the actual data pattern
 - This is the whole point of scaling — preserve the information, just change the scale
- Before Scaling (Left Plot)

- x-axis ranges from 0 to 2,500 — these are the original capped values after IQR treatment
- The distribution is right-skewed with most trips clustered between 0 and 500
- There is a spike at the far right around 2,400 — this is the IQR upper cap, all outlier values were clipped to this point so they pile up there
- The KDE curve shows two bumps — one at the left peak and one at the right cap spike
- After Scaling (Right Plot)
 - x-axis now ranges from -1.0 to 2.0 — same data, just rescaled
 - Mean is now at 0.0 and std is 1.0 — confirmed by the describe() output earlier
 - The spike at the right is now at ~2.05 instead of 2,400 — same values, different scale
 - The left cluster is now between -1.0 and 0 — negative values do not mean anything is wrong, they simply mean those trips were below the mean
- The Spike at the Right End
 - Both plots show an unusual spike at the maximum value
 - This is a direct result of the IQR capping done earlier — all values above the upper limit were clipped to the same number, creating a pile-up at that point
 - This is expected and acceptable behaviour after outlier treatment using clip()

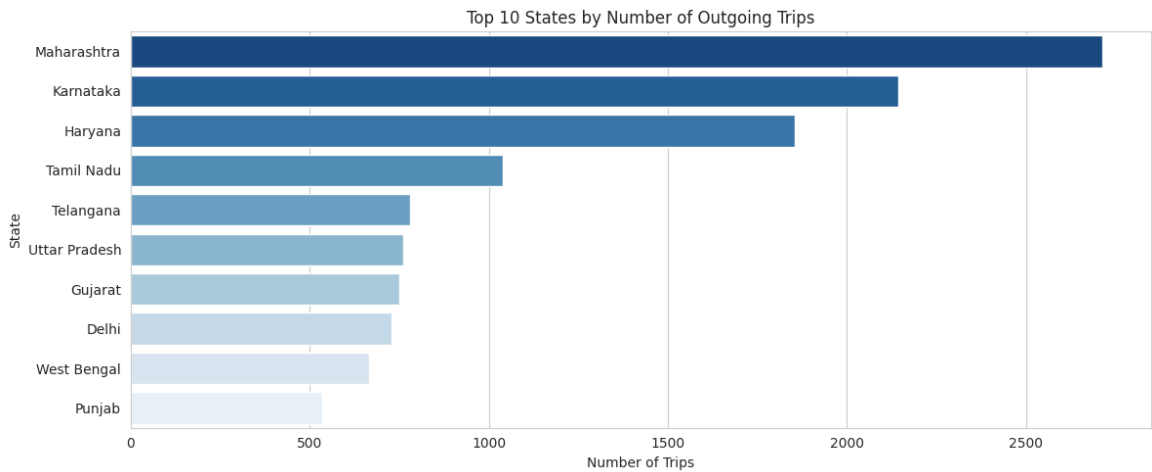
13. Business Insights & Recommendations

```
In [45]: # states that send the most number of trips

top_states = trip['source_state'].value_counts().head(10)

plt.figure(figsize=(12, 5))
sns.barplot(x=top_states.values, y=top_states.index, palette='Blues')
plt.title('Top 10 States by Number of Outgoing Trips')
plt.xlabel('Number of Trips')
plt.ylabel('State')
plt.tight_layout()
plt.show()

# INSIGHT: These are the most active source states.
# Delhivery should prioritise warehouse capacity and staff in these
```



Observations:

- Maharashtra, Karnataka and Haryana are Delhivery's three most active source states, together generating the bulk of all outgoing trips.
- The sharp drop from the top 3 to the rest of the top 10 highlights a concentration risk — if operations in these three states face disruption, the overall network would be significantly impacted.
- Delhivery should prioritise infrastructure, staffing and backup capacity in these states above all others.

In [46]: *# Busiest Corridors (State to State)*

```
corridor_stats = trip.groupby('corridor').agg(
    Number_of_Trips      = ('trip_uuid',          'count'),
    Avg_Distance_km      = ('actual_distance_to_destination', 'mean'),
    Avg_Actual_Time      = ('actual_time',         'mean'),
    Avg_OSRM_Time        = ('osrm_time',          'mean')
).sort_values('Number_of_Trips', ascending=False).reset_index()

print('Top 15 Busiest Corridors:')
print(corridor_stats.head(15).to_string(index=False))
```

Top 15 Busiest Corridors:

	corridor	Number_of_Trips	Avg_Distance_km
Avg_Actual_Time	Avg_OSRM_Time		
874.672646	Maharashtra to Maharashtra	2453	30.647221
388.253975			
437.869163	Karnataka to Karnataka	2056	23.418124
233.361868			
387.816846	Tamil Nadu to Tamil Nadu	1021	23.601801
213.538688			
10203.351522	Haryana to Haryana	953	69.295441
5576.034627			
636.560584	Telangana to Telangana	685	29.432303
312.109489			
606.647059	Gujarat to Gujarat	629	33.413774
334.521463			
1796.447154	West Bengal to West Bengal	615	33.040919
722.352846			
1744.683417	Uttar Pradesh to Uttar Pradesh	597	43.843818
785.070352			
4303.536424	Rajasthan to Rajasthan	453	59.217820
2153.273731			
1625.956522	Andhra Pradesh to Andhra Pradesh	414	47.916325
880.355072			
1000.276961	Punjab to Punjab	408	49.265467
478.536765			
428.574307	Delhi to Haryana	397	27.190761
188.254408			
1034.707965	Bihar to Bihar	339	43.115569
302.398230			
457.592949	Haryana to Delhi	312	29.360959
220.596154			
4998.321033	Madhya Pradesh to Madhya Pradesh	271	84.419354
2437.712177			



Observations:

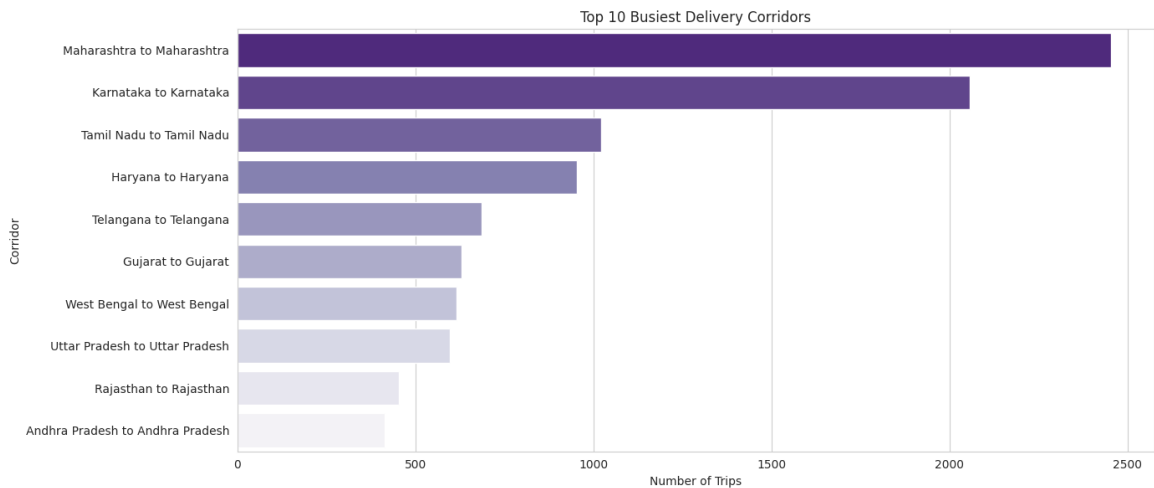
- 13 of the top 15 busiest corridors are intra-state, confirming Delhivery's hub-and-spoke distribution model where goods are moved between local hubs before travelling across states.
- Maharashtra, Karnataka and Tamil Nadu dominate trip volume with short average distances under 31 km.
- The most critical finding is Haryana's intra-state corridor which despite moderate trip volume has an average actual time of 10,203 — nearly 10x higher than comparable corridors — pointing to a serious operational inefficiency that requires immediate investigation.

```
In [53]: # Bar chart of top 10 corridors

plt.figure(figsize=(14, 6))
sns.barplot(data=corridor_stats.head(10), x='Number_of_Trips', y='c
plt.title('Top 10 Busiest Delivery Corridors')
plt.xlabel('Number of Trips')
plt.ylabel('Corridor')
```



```
plt.tight_layout()
plt.show()
```



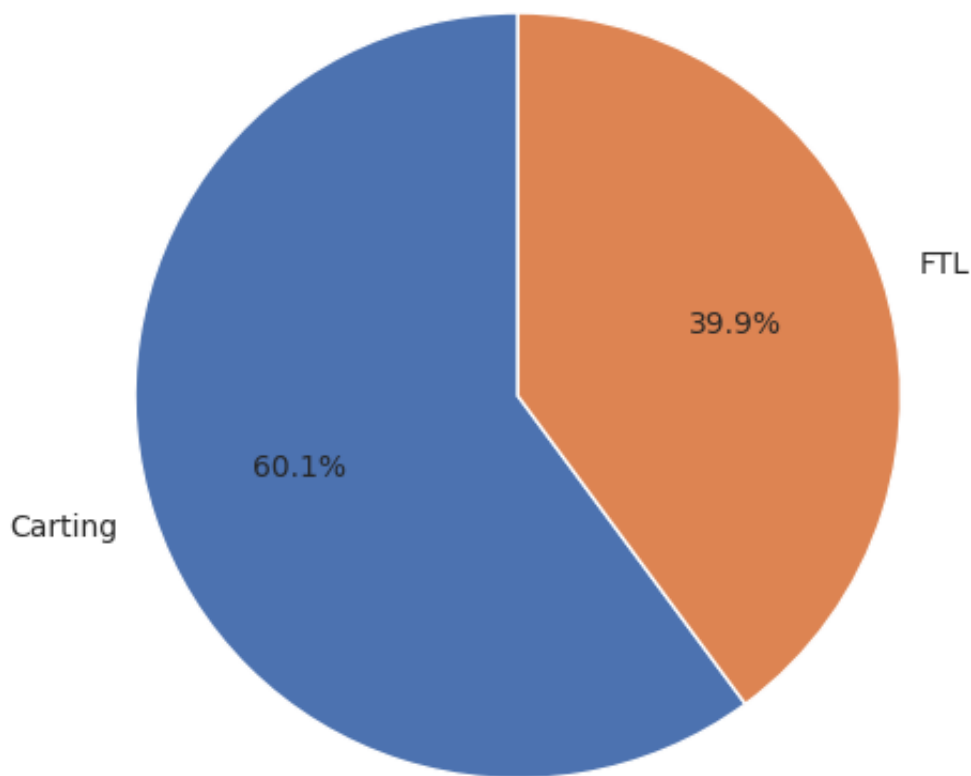
Observations:

- Every single bar in this top 10 chart represents a same-state corridor — source and destination are identical
- This is immediately visible just by reading the y-axis labels — every entry says "X to X"
- This visually reinforces the hub-and-spoke finding from the table analysis
- The bar chart makes the network concentration immediately visible — Maharashtra and Karnataka together generate nearly 4,500 trips while the remaining 8 corridors combined account for roughly the same number.
- Every corridor in the top 10 is intra-state, visually confirming Delhivery's hub-and-spoke model.
- The long tail from rank 3 onwards suggests significant room to grow volume in states like Rajasthan and Andhra Pradesh where Delhivery currently has a lighter footprint.

In [54]: *# Route Type Split*

```
plt.figure(figsize=(6, 5))
trip['route_type'].value_counts().plot.pie(
    autopct='%1.1f%%',
    startangle=90,
    colors=['#4C72B0', '#DD8452']
)
plt.title('Route Type Distribution')
plt.ylabel('')
plt.tight_layout()
plt.show()
# INSIGHT: FTL = long-distance, full truck. Carting = short-range,
```

Route Type Distribution



Observations:

- Carting trips at 60.1% outnumber FTL trips at 39.9%, confirming that Delhivery's network is primarily built around short-range last-mile transfers between hubs — consistent with the intra-state dominance seen in the corridor analysis.
- FTL's 40% share still represents a substantial long-haul operation that requires dedicated large vehicle capacity.
- Managing both fleet types efficiently is critical since neither route type can be deprioritised without impacting the overall delivery network.

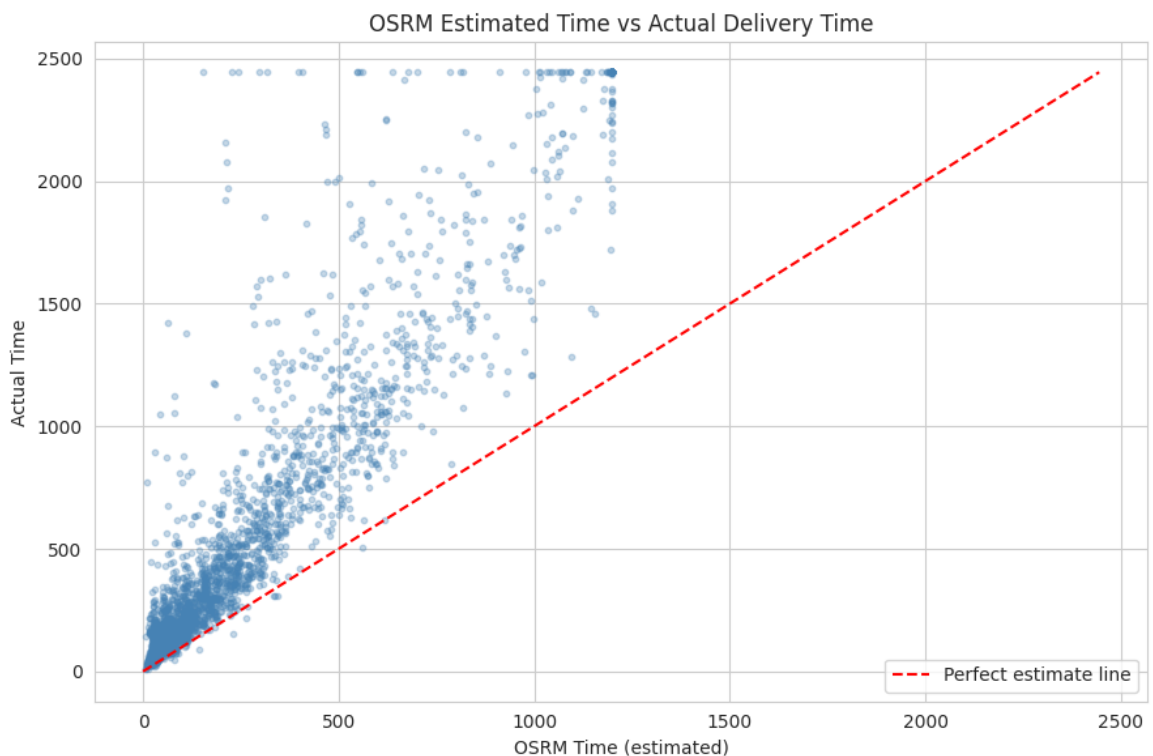
```
In [60]: # OSRM vs Actual Time Scatter Plot
# Points on the red line = OSRM was perfectly accurate
# Points ABOVE the line = actual trip took longer than expected

sample = trip_clean.sample(3000, random_state=42)

plt.figure(figsize=(9, 6))
plt.scatter(sample['osrm_time'], sample['actual_time'], alpha=0.3,

max_val = max(sample['osrm_time'].max(), sample['actual_time'].max())
plt.plot([0, max_val], [0, max_val], 'r--', label='Perfect estimate
```

```
plt.xlabel('OSRM Time (estimated)')
plt.ylabel('Actual Time')
plt.title('OSRM Estimated Time vs Actual Delivery Time')
plt.legend()
plt.tight_layout()
plt.show()
```



Observations:

- The scatter plot visually confirms that OSRM underestimates actual delivery time for the vast majority of trips — almost all points lie above the perfect estimate line.
- The underestimation gap widens as trip length increases, meaning OSRM is least reliable for long distance FTL routes.
- The horizontal and vertical bands of capped points at the chart boundaries are a direct result of IQR outlier treatment and do not represent actual trips.
- For reliable ETA predictions, Delhivery must apply a correction multiplier on top of OSRM estimates, particularly for longer routes.

```
In [63]: # Correlation Heatmap
# Shows how strongly columns are related to each other.
# +1 = perfectly correlated, 0 = no relation, -1 = opposite

corr_columns = [
    'actual_distance_to_destination',
    'actual_time',
    'osrm_time',
    'osrm_distance',
    'od_time_diff_hour',
```

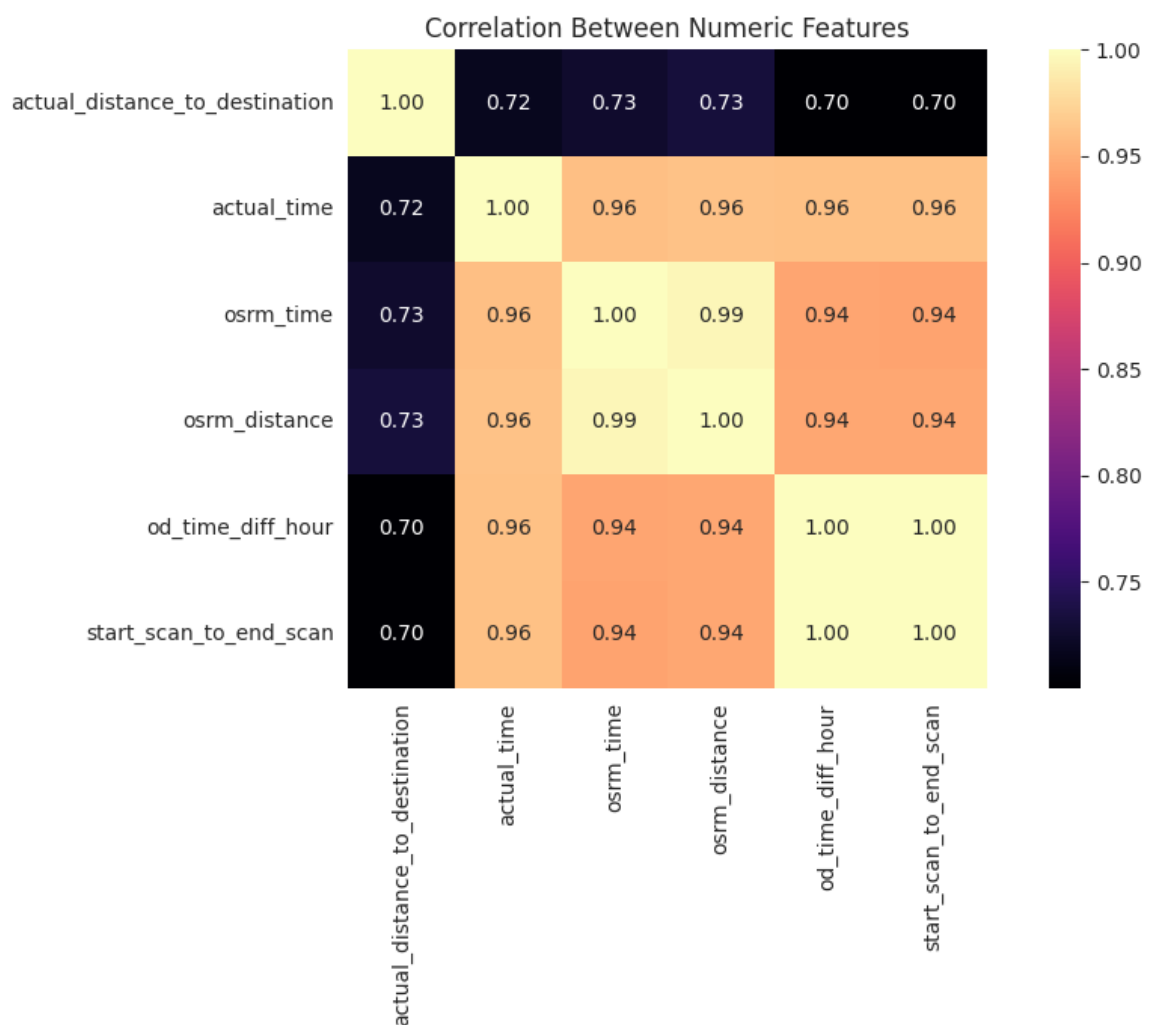
```

        'start_scan_to_end_scan'
    ]

plt.figure(figsize=(10, 7))
sns.heatmap(
    trip_clean[corr_columns].corr(),
    annot=True,
    fmt='.2f',
    cmap='magma',
    square=True
)
plt.title('Correlation Between Numeric Features')
plt.tight_layout()
plt.show()

# INSIGHT: actual_time and osrm_time are highly correlated – OSRM i

```



Observations:

- All six features are strongly positively correlated with each other, with values ranging from 0.70 to 0.99.
- The near-perfect correlation between osrm_time and osrm_distance (0.99) and between od_time_diff_hour and start_scan_to_end_scan (1.00) confirms these pairs are measuring the same underlying

phenomenon through different systems.

- While this validates data consistency, the high multicollinearity also means redundant columns should be dropped before building forecasting models to avoid inflated model complexity without added predictive value.



Business Insights Summary

What the data tells us:

- **Maharashtra, Karnataka and Haryana** generate the majority of outgoing trips — these three states alone dominate Delhivery's logistics network and are the most critical markets to protect and grow.
- **OSRM underestimates actual delivery time by nearly 2x** (mean OSRM: 2,091 vs mean actual: 4,076) — real trips consistently take longer than the routing engine predicts due to traffic, loading delays and route diversions.
- **All top 10 busiest corridors are intra-state** — Delhivery operates a hub-and-spoke model where goods move between local hubs before crossing state borders. Maharashtra to Maharashtra alone accounts for 2,453 trips.
- **Wednesday is the peak day with 26,732 trips** while Sunday is the slowest with 17,870 — a 33% drop, showing clear weekly patterns in delivery demand.
- **Haryana's intra-state corridor has an abnormally high average delivery time of 10,203** compared to Karnataka's 437 and Tamil Nadu's 387 — pointing to a serious operational issue specific to that corridor.



Recommendations:

Action

1. **Add at least 50% buffer to OSRM estimates** for customer ETAs — the 2x gap between OSRM and actual time means even a 15-20% buffer is not enough for most trips.
2. **Prioritise infrastructure in Maharashtra, Karnataka and Haryana** — more warehouses, staff and vehicles in these three states will have the highest impact on overall network performance.
3. **Urgently investigate Haryana's intra-state deliveries** — an average time of 10,203 against a corridor distance of only 69 km is a major red flag that needs immediate attention.
4. **Schedule vehicle maintenance on Sundays** — with 33% fewer trips than Wednesday, Sunday is the safest day to take vehicles off the road without

impacting customers.

5. **Investigate very long delivery trips** — extreme outliers reaching 167,920 minutes represent stuck parcels, failed deliveries or data recording errors that are hurting average performance metrics.
6. **Standardise scanning practices across all hubs** — the perfect 1.00 correlation between `od_time_diff_hour` and `start_scan_to_end_scan` but massive scale difference suggests two systems recording the same event in different units, which must be standardised.